Java Programlama – Sınav Çalışma Notu

İçindekiler

- Kapsam
- PDF 1 · Variables ve Data Types
- PDF 1 · Operators ve Control Statements
- PDF 2 · if ve switch
- PDF 2 · for ve for-each
- PDF 3 · while, do-while, break, continue, comments
- PDF 4 · OOP kavramları ve naming convention
- PDF 4 · Class & Object, Methods, Constructors
- PDF 5 · static ve this
- PDF 5 · Overloading, Overriding, super
- PDF 6 · Inheritance ve Aggregation
- Ek çıktı tahmini soruları
- Sık yapılan hatalar
- Son tekrar listesi

Kapsam

Bu not yalnızca Dr. Öğr. Üyesi Baki Gökgöz'ün 6 ders PDF'ini kapsar. Her kodun altında **ne işe yaradığı**, alıştırmaların cevabı ve sık yapılan hatalar var.

PDF	Konu
1	Variables, Data Types, Operators, Control Statements'a giriş
2	if-else, switch, for, for-each
3	while, do-while, break, continue, comments
4	OOP kavramları, naming convention, class/object, methods, constructors
5	this, static, method overloading, method overriding, super
6	Inheritance (IS-A), aggregation (HAS-A)

Okuma kuralı: Kod bloğunun hemen altındaki **Ne yapıyor?** maddeleri kodu satır satır açıklar; **Çıktı** her zaman PDF'teki çıktıdır.

PDF 1 · Variables ve Data Types

Variable: RAM'de ayrılmış bir alana verilen isim ("vary + able": değeri değişebilir). Java **statically typed**: her variable'ın tipi compile time'da bilinir. Data type, ne tür veri tutulacağını ve bellekte ne kadar yer kaplayacağını belirler.

`int data = 50;` → int = data type · data = identifier · = assignment · 50 = literal

Declaration, initialization, assignment

```java
int age;                    // declaration
age = 21;                   // assignment
int year = 2026;            // declaration + initialization
int a = 5, b = 10;          // tek satırda birden fazla
final double PI = 3.14159;  // constant
var city = "Gümüşhane";     // Java 10+: type inference
age = age + 1;
System.out.println(age + " " + year);  // 22 2026
System.out.println(a + b);             // 15
System.out.println(PI + " " + city);   // 3.14159 Gümüşhane
// PI = 3.14; -> compile error
```

Kod	Ne yapıyor?
int age;	**Declaration**: tip + isim, bellekte yer ayrılır
age = 21;	**Assignment**: değer atanır
int year = 2026;	**Initialization**: ilk değer tanımla birlikte verilir
final double PI	**Sabit**: yalnızca bir kez atanır, tekrar atama compile error. Sabitler UPPER_CASE
var city	Tipi sağ taraftan çıkarır (String). Java yine statically typed: city sonradan başka tip alamaz

Identifier kuralları

Zorunlu kural	Konvansiyon (öneri)
Harf, rakam, _ ve $ içerebilir	variable / method: camelCase → studentName
Rakamla başlayamaz	class: PascalCase → Student
Boşluk ve - içeremez	constant: UPPER_SNAKE_CASE → MAX_SIZE
Reserved keyword olamaz (int, class, for)	Anlamlı isim: x yerine totalPrice
Case-sensitive: age ≠ Age	—

Geçerli: count, _temp, $price, student2 · **Geçersiz:** 2student (rakam), my-var (tire), class (keyword), first name (boşluk)

3 tür variable

Özellik	Local	Instance	Static
Tanımlandığı yer	Method / block içinde	Class içinde, method dışında	Class içinde, static ile
Oluşma	Method/block çalışınca	new ile object oluşunca	Class yüklenince (bir kez)
Kopya sayısı	Her çağrıda yeni	Her object'e bir	Tüm class için tek
Default değer	**Yok** (initialize zorunlu)	0, false, null	0, false, null
Erişim	Sadece kendi scope'u	obj.name	ClassName.name
Ömür	Block bitene kadar	Object erişilebilir kaldıkça	Class yüklü kaldıkça

Java'da **global variable yoktur**; çok yerden erişilecek veri static field olur. Local variable **static olamaz**.

Local variable ve scope

```
int num = 10;                       // local
System.out.println("num: " + num); // num: 10
for (int i = 0; i < 2; i++) {
    int square = i * i;             // sadece bu block'ta
    System.out.println("square: " + square); // square: 0, square: 1
}
// System.out.println(square);     // HATA: scope dışı
int total;
// System.out.println(total);      // HATA: initialize edilmemiş
total = 5;
System.out.println("total: " + total);  // total: 5
```

Ne yapıyor? square ve i, for block'u bitince yok olur. Değer verilmemiş local variable kullanılırsa "might not have been initialized" hatası.

Instance variable

```
class Student {
    String name;          // instance
    int age = 19;         // instance
    Student(String name) { this.name = name; }
}
Student s1 = new Student("Ali");
Student s2 = new Student("Ayşe");
s2.age = 21;
System.out.println(s1.name + " - " + s1.age);  // Ali - 19
System.out.println(s2.name + " - " + s2.age);  // Ayşe - 21
```

Ne yapıyor? s1 ve s2 ayrı object; s2.age değişince s1.age etkilenmez. this.name object'in field'ı, name ise constructor parametresi. Bir .java dosyasında yalnızca **bir public class** olabilir.

Static variable

```java
class Student {
    static int count = 0;   // paylaşılır
    String name;
    Student(String name) { this.name = name; count++; }
}
Student s1 = new Student("Ali");
Student s2 = new Student("Ayşe");
Student s3 = new Student("Mehmet");
System.out.println("Toplam: " + Student.count);  // Toplam: 3
s1.count = 99;                                   // önerilmez
System.out.println(s2.count + " " + s3.count);    // 99 99
```

Ne yapıyor? count bellekte tek yerde; her new'de constructor onu artırır. s1.count = 99 yapınca herkes 99 görür. Doğru erişim: Student.count. Tipik kullanım: sayaç, sabit (static final).

Data type hiyerarşisi

Grup	Türler	Ne tutar
Primitive (8 tane)	boolean · char · byte, short, int, long · float, double	**Değerin kendisini**
Non-primitive (reference)	String, Array, Class, Interface, Enum	Object'in **adresini**

8 primitive type

Type	Boyut	Aralık	Default	Literal
byte	8 bit	−128 ... 127	0	(byte) 100
short	16 bit	−32.768 ... 32.767	0	30000
int	32 bit	−2.147.483.648 ... 2.147.483.647	0	42, 0xFF
long	64 bit	≈ ±9,22 × 10^{18}	0L	15000000000L
float	32 bit	≈ ±3,4 × 10^{38} (6-7 basamak)	0.0f	3.14f
double	64 bit	≈ ±1,8 × 10^{308} (15-16 basamak)	0.0	3.14, 2e3
char	16 bit	'\u0000' ... '\uffff' (0 ... 65.535)	'\u0000'	'A'
boolean	JVM'e bağlı*	true / false	false	true

*JLS boolean için boyut tanımlamaz; çoğu JVM 1 byte kullanır. Default değerler yalnızca field'lar ve array elemanları içindir.

Integer type'lar

```java
byte b = 100;
short s = 30_000;                   // _ okunabilirlik (Java 7+)
int population = 85_000_000;
long distance = 15_000_000_000L;    // L eki şart
int hex = 0xFF;                     // 255
int bin = 0b1010;                   // 10
int max = Integer.MAX_VALUE;        // 2147483647
System.out.println(max + 1);        // -2147483648 (overflow)
System.out.println(Long.MAX_VALUE); // 9223372036854775807
```

Ne yapıyor? L eki olmazsa sayı int sayılır → "integer number too large". **Overflow hata vermez**, sessizce başa sarar. Büyük sayılar için long veya BigInteger.

float ve double

```java
float f = 234.5f;               // f eki zorunlu
double d = 12.3;                // varsayılan ondalık tip
double sci = 1.5e3;             // 1500.0
System.out.println(10 / 4);       // 2   (int / int)
System.out.println(10 / 4.0);     // 2.5
System.out.println(1.0f / 3);     // 0.33333334
System.out.println(1.0 / 3);      // 0.3333333333333333
System.out.println(0.1 + 0.2);    // 0.30000000000000004
System.out.println(0.1 + 0.2 == 0.3); // false
```

Ne yapıyor? Ondalık literal varsayılan **double**: float f = 3.5; → compile error. IEEE 754 formatı 0.1'i tam temsil edemez; para hesabında BigDecimal kullan, ondalıkları == ile karşılaştırma.

char ve boolean

```java
char c = 'A';
char u = '\u0041';                  // yine 'A'
char tr = 'ş';
System.out.println(c + " " + u + " " + tr);  // A A ş
System.out.println(c + 1);          // 66  (int'e dönüşür)
System.out.println((char) (c + 1));  // B
c++;
System.out.println(c);              // B
System.out.println((int) 'a');      // 97
boolean isPassed = 75 >= 50;
boolean isEmpty = false;
System.out.println(isPassed + " " + !isEmpty); // true true
```

Ne yapıyor? char 16-bit **Unicode** (ASCII değil), bu yüzden 2 byte. Aritmetikte char önce int'e yükselir. boolean yalnızca true/false: boolean b = 1; → HATA.

Type casting

Widening (otomatik): byte → short → int → long → float → double (char → int). Veri kaybı yok; yalnız long → float/double'da hassasiyet kaybı olabilir.

Narrowing (açık cast): (type) yazmak gerekir; kesirli kısım atılır, aralık dışı değer taşar. **boolean hiçbir tipe cast edilemez.**

```java
int a = 10;
double d = a;                       // widening → 10.0
double price = 10.99;
int p = (int) price;                // narrowing → 10
System.out.println(Math.round(price)); // 11
int big = 130;
byte b = (byte) big;                // -126
byte x = 10, y = 20;
// byte z = x + y;                  // HATA: x + y int olur
byte z = (byte) (x + y);            // 30
```

Satır	Ne yapıyor?
double d = a	Küçükten büyüğe otomatik
(int) price	Kesirli kısmı **atar** (truncate), yuvarlamaz
Math.round(price)	Yuvarlama yapar
(byte) 130	byte aralığı −128...127; 130 − 256 = **−126** (two's complement)
x + y	byte, short, char aritmetikte önce **int'e yükselir** (numeric promotion)

Reference type'lar

Değerin kendisini değil object'in **adresini** tutar (primitive değer stack'te doğrudan, reference adres olarak). Default değeri **null**; null üzerinde method çağırmak → **NullPointerException**. İsimleri büyük harfle başlar (String, Scanner, Integer).

```java
enum Color { RED, GREEN, BLUE }
String str = "Merhaba";
int[] arr = {1, 2, 3, 4, 5};
Color clr = Color.BLUE;
System.out.println(str.length());        // 7
System.out.println(str.toUpperCase());   // MERHABA
System.out.println("Boyut: " + arr.length); // Boyut: 5
System.out.println(clr);                 // BLUE
String s2 = null;
System.out.println(s2 == null);          // true
```

Kod	Ne yapıyor?
str.length()	String uzunluğu: **method** (parantezli)
arr.length	Array uzunluğu: **field** (parantezsiz)
str.toUpperCase()	**Yeni** String döndürür; String **immutable**, str değişmez
enum Color	Sabit değerler kümesi (gün, renk, durum)

Wrapper class ve autoboxing

```java
Integer x = 42;                         // autoboxing: int -> Integer
int y = x;                              // unboxing: Integer -> int
int n = Integer.parseInt("123");        // String -> int
double r = Double.parseDouble("2.5");   // String -> double
String s = String.valueOf(3.5);         // double -> String
System.out.println(n + 1);              // 124
System.out.println(r * 2);              // 5.0
System.out.println(s + 1);              // 3.51 (String birleştirme)
System.out.println(Integer.MAX_VALUE);  // 2147483647
System.out.println(Character.isDigit('7'));     // true
System.out.println(Integer.toBinaryString(y));  // 101010
```

Metot	Ne yapıyor?
Integer.parseInt("123")	Metni int'e çevirir
Double.parseDouble("2.5")	Metni double'a çevirir
String.valueOf(3.5)	Sayıyı metne çevirir
Character.isDigit('7')	Karakter rakam mı?
Integer.toBinaryString(42)	İkilik gösterim (101010)

Her primitive'in wrapper'ı var: int → **Integer**, char → **Character**, diğerleri baş harfi büyük (Double, Boolean...). ArrayList<Integer> gibi collection'lar primitive tutamaz, wrapper gerekir.

PDF 1 · Operators ve Control Statements'a giriş

Kategori	Operator'ler	Ne yapar
Unary	+ - ++ -- ! ~	Tek operand alır
Arithmetic	+ - * / %	Temel matematik
Relational	< > <= >= == !=	boolean döndürür
Logical	&& || !	Koşulları birleştirir
Bitwise	& | ^ ~	Bit düzeyinde işlem
Shift	<< >> >>>	Bitleri kaydırır
Assignment	= += -= *= /= %=	Değer atar
Ternary	c ? a : b	Tek satırlık if-else

Arithmetic

```java
int a = 10, b = 3;
System.out.println(a + b);      // 13
System.out.println(a - b);      // 7
System.out.println(a * b);      // 30
System.out.println(a / b);      // 3   integer division
System.out.println(a % b);      // 1   kalan
System.out.println(-10 % 3);    // -1  işaret soldan gelir
System.out.println(10.0 / 3);   // 3.3333333333333335
System.out.println(7.5 % 2);    // 1.5
System.out.println(10.0 / 0);   // Infinity
// System.out.println(10 / 0);  // ArithmeticException
```

Ne yapıyor? int / int → kesirli kısım atılır. % kalanı verir; sonucun işareti **soldaki** sayıdan gelir. Çift/tek kontrolü: n % 2 == 0. **10 / 0 → ArithmeticException**, ama **10.0 / 0 → Infinity**.

Unary: ++ ve --

```java
int x = 10;
System.out.println(+x);     // 10
System.out.println(-x);     // -10
System.out.println(x++);    // 10  önce kullan, sonra artır (x=11)
System.out.println(x);      // 11
System.out.println(++x);    // 12  önce artır, sonra kullan
System.out.println(x--);    // 12  önce kullan, sonra azalt (x=11)
System.out.println(--x);    // 10  önce azalt, sonra kullan
boolean ok = true;
System.out.println(!ok);    // false  logical complement
System.out.println(~5);     // -6     bitwise complement
```

İfade	Ne yapar
x++ (post)	İfadede **eski** değer kullanılır, sonra artar
++x (pre)	Önce artar, **yeni** değer kullanılır
!ok	boolean'ı ters çevirir
~x	Her biti ters çevirir: **~x = −x − 1** (~5 = −6)

Relational ve logical

```
int a = 10, b = 20, c = 5;
System.out.println(a < b);              // true
System.out.println(a == b);             // false
System.out.println(a != b);             // true
System.out.println(a < b && b > c);     // true
System.out.println(a > b || a > c);     // true
System.out.println(!(a < b));           // false
if (a < c && ++a > 0) { }               // a < c false → ++a ÇALIŞMAZ
System.out.println("a = " + a);         // a = 10
if (a < c & ++a > 0) { }                // & iki tarafı da çalıştırır
System.out.println("a = " + a);         // a = 11
```

Ne yapıyor? Short-circuit: && ilk koşul false ise ikinciye bakmaz; || ilk koşul true ise ikinciye bakmaz. & ve | short-circuit **yapmaz**. null kontrolü &&'nin **soluna** yazılır.

A	B	A && B	A || B	!A	A ^ B
true	true	true	true	false	false
true	false	false	true	false	true
false	true	false	true	true	true
false	false	false	false	true	false

Kural: && ikisi de true ise true · || en az biri true ise true · ^ iki taraf **farklıysa** true. ! logical NOT (boolean), ~ bitwise complement (bitler).

Bitwise ve shift

İşlem	Hesap	Sonuç
10 | 5	1010 | 0101	1111 = 15
11 & 5	1011 & 0101	0001 = 1
11 ^ 5	1011 ^ 0101	1110 = 14
~5	−5 − 1	−6

Kullanım: flag bitleri, (n & 1) == 0 ile çift kontrolü, maskeleme, hash/şifreleme.

```
System.out.println(10 << 2);    // 40   10 * 2^2
System.out.println(15 << 4);    // 240  15 * 2^4
System.out.println(20 >> 2);    // 5    20 / 2^2
System.out.println(20 >> 3);    // 2    20 / 2^3 (aşağı)
System.out.println(-20 >> 2);   // -5   işaret korunur
System.out.println(-20 >>> 2);  // 1073741819  soldan 0 gelir
System.out.println(Integer.toBinaryString(20));      // 10100
System.out.println(Integer.toBinaryString(20 >> 2)); // 101
```

Operator	Ne yapar
x << n	≈ x × 2^n
x >> n	≈ x ÷ 2^n (aşağı yuvarlar); **signed**: soldan işaret biti kopyalanır
x >>> n	**unsigned**: soldan 0 gelir; negatif sayı büyük pozitife döner

Assignment

```java
int a = 10;
a += 3;  // 13   a = a + 3
a -= 4;  // 9
a *= 2;  // 18
a /= 4;  // 4
a %= 3;  // 1
short s = 10;
s += 5;              // OK: gizli cast içerir
// s = s + 5;        // HATA: s + 5 int olur
s = (short) (s + 5); // 20
int x, y, z;
x = y = z = 7;       // sağdan sola
System.out.println(x + y + z); // 21
```

Ne yapıyor? a op= b aslında **a = (T)(a op b)**: içinde gizli cast var, bu yüzden s += 5 geçerli ama s = s + 5 hata. Assignment **sağdan sola** çalışır. Widening otomatik: double d = 5; geçerli.

Ternary (? :)

```java
int a = 2, b = 5;
int min = (a < b) ? a : b;                    // min: 2
int x = 10, y = 15, z = 13;
int max = (x > y) ? (x > z ? x : z) : (y > z ? y : z);  // max: 15
int number = 13;
String type = (number % 2 == 0) ? "çift" : "tek";      // 13 sayısı tek
int score = 72;
System.out.println(score >= 50 ? "Geçti" : "Kaldı");  // Geçti
```

Ne yapıyor? condition ? trueValue : falseValue. Java'daki **tek üç operand'lı** operator; bir değer üretir. İç içe ternary okunabilirliği düşürür; karmaşık koşulda if-else kullan.

Operator precedence

Öncelik	Operator'ler	Yön
1 Postfix	expr++ expr--	→
2 Unary	++expr --expr + - ~ !	←
3 Multiplicative	* / %	→
4 Additive	+ -	→
5 Shift	<< >> >>>	→
6 Relational	< > <= >= instanceof	→
7 Equality	== !=	→
8 Bitwise	& ^ | (bu sırayla)	→
9 Logical	&& || (bu sırayla)	→
10 Ternary	? :	←
11 Assignment	= += -= ...	←

Örnek: `10*10/5+3-1*4/2` → önce * ve / soldan sağa: `100/5 = 20`, `1*4/2 = 2` → sonra `20 + 3 − 2` = **21**. Emin değilsen parantez kullan.

Control statement türleri

Varsayılan akış yukarıdan aşağı (sequential); control statement'lar bunu değiştirir.

Decision making	Loop	Jump
if, if-else, if-else-if ladder, nested if, switch	for, for-each, while, do-while	break, continue, return

```java
int sum = 0;
for (int j = 1; j <= 10; j++) { sum = sum + j; }
System.out.println("1..10 toplamı: " + sum);   // 55
if (sum > 50) System.out.println("Toplam 50'den büyük");
else System.out.println("Toplam 50 veya daha küçük");
int i = 0;
while (i <= 10) { System.out.print(i + " "); i += 2; }  // 0 2 4 6 8 10
```

Ne yapıyor? for 1'den 10'a toplar; if toplamı kontrol eder; while 0'dan 10'a çiftleri yazar. for başlığındaki üç kısım **noktalı virgülle** ayrılır (kaynaktaki virgüllü syntax hatalı). print satır atlamaz, println atlar.

String karşılaştırma: == mi equals() mi?

```
String a = "Java";
String b = new String("Java");
String c = "Java";
String d = "JAVA";
System.out.println(a == b);                 // false  farklı object
System.out.println(a == c);                 // true   String pool (tesadüf)
System.out.println(a.equals(b));            // true   içerik aynı
System.out.println(a.equalsIgnoreCase(d));  // true   harf duyarsız
```

Ne yapıyor? == **reference'ı** (adres), equals() **içeriği** karşılaştırır. a == c true çünkü literal'lar String pool'dan gelir. Scanner girdisinde == yanlış sonuç verir; **her zaman equals()**.

PDF 1 alıştırma cevapları

S1. System.out.println("Toplam: " + 2 + 3); ve System.out.println(2 + 3 + " Toplam");

- Cevap: Toplam: 23 / 5 Toplam
- Neden: Soldan sağa. İlkinde önce String var, 2 ve 3 metne eklenir. İkincide önce 2 + 3 = 5 hesaplanır, sonra metne eklenir.

S2. int i = 5; int j = i++ + ++i; System.out.println(i + " " + j);

- Cevap: 7 12
- Neden: i++ → 5 kullanılır, i = 6. ++i → i = 7, 7 kullanılır. j = 5 + 7 = 12.

S3. `System.out.println(7 / 2 * 2.0);` ve `System.out.println(7 / 2.0 * 2);`

- Cevap: 6.0 / 7.0
- Neden: 7 / 2 = 3 (integer division), 3 * 2.0 = 6.0. 7 / 2.0 = 3.5, 3.5 * 2 = 7.0.

S4. byte b = (byte) 200; char c = (char) ('a' + 2); System.out.println(b + " " + c);

- Cevap: -56 c
- Neden: 200 byte'a sığmaz: 200 − 256 = −56. 'a' = 97, 97 + 2 = 99 = 'c'.

PDF 2 · if Statements ve switch

Program bir koşulu değerlendirir, sonuca göre hangi kodun çalışacağına karar verir. Koşul **mutlaka boolean** üretmeli. Java'da 4 tür if var: **simple if, if-else, if-else-if ladder, nested if**. Tek satırlık body'de { } zorunlu değil ama her zaman kullanılması önerilir.

Simple if

```
int age = 20;
if (age > 18) { System.out.println("Yaş 18'den büyük"); }       // çalışır
int temperature = 35;
if (temperature > 30) { System.out.println("Hava sıcak, su için!"); } // çalışır
if (temperature < 0) { System.out.println("Don uyarısı!"); }   // false: atlanır
System.out.println("Program devam ediyor");
```

Ne yapıyor? Koşul true ise block çalışır, false ise atlanır ve program sonraki satırdan devam eder. Kullanım: doğrulama, uyarı, tek yönlü kontrol.

if-else

```
int number = 13;
if (number % 2 == 0) { System.out.println(number + " çift sayıdır"); }
else { System.out.println(number + " tek sayıdır"); }   // 13 tek sayıdır
```

Ne yapıyor? number % 2 kalanı 0 ise çift, değilse tek. İki yoldan **tam biri** çalışır.

Leap year (artık yıl):

```java
int[] years = {2020, 2023, 1900, 2000};
for (int year : years) {
    boolean leap = (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
    if (leap) System.out.println(year + " -> LEAP YEAR");
    else System.out.println(year + " -> COMMON YEAR");
}
// 2020 -> LEAP · 2023 -> COMMON · 1900 -> COMMON · 2000 -> LEAP
```

Ne yapıyor? Kural: 4'e bölünüp 100'e bölünmeyen **veya** 400'e bölünen yıl artık yıldır. 1900: 4 ve 100'e bölünür ama 400'e bölünmez → artık yıl değil. Karmaşık koşulu boolean variable'a (leap) atamak okunabilirliği artırır.

if-else-if ladder

Koşullar yukarıdan aşağı kontrol edilir; **ilk true olan** çalışır, gerisi atlanır. Hiçbiri true değilse en sondaki else çalışır. Kullanım: not sistemi, vergi dilimi, menü.

```java
int marks = 65;
if (marks < 0 || marks > 100) System.out.println("Geçersiz not!");
else if (marks >= 90) System.out.println("A+ grade");
else if (marks >= 80) System.out.println("A grade");
else if (marks >= 70) System.out.println("B grade");
else if (marks >= 60) System.out.println("C grade");   // çalışır
else if (marks >= 50) System.out.println("D grade");
else System.out.println("fail");
```

Ne yapıyor? Önce geçersiz değerler elenir, sonra **büyükten küçüğe** kontrol edilir; bu sayede marks >= 60 && marks < 70 gibi tekrarlı koşullar gerekmez. (Kaynakta marks < 100 yazdığı için 100 puan "Invalid" çıkıyordu; düzeltilmiş.)

Sıra önemli:

```java
int marks = 95;
if (marks >= 50) System.out.println("D grade");        // ✗ çalışır: D grade
else if (marks >= 90) System.out.println("A+ grade");  // hiç kontrol edilmez
```

Ne yapıyor? 95 ≥ 50 true → ilk block çalışır. Bu bir **logic error**: derleyici uyarmaz. Doğrusu: en kısıtlayıcı koşul (>= 90) önce.

Positive / negative / zero:

```java
int[] numbers = {-13, 0, 42};
for (int number : numbers) {
    if (number > 0) System.out.println(number + ": POSITIVE");
    else if (number < 0) System.out.println(number + ": NEGATIVE");
    else System.out.println(number + ": ZERO");
}
// -13: NEGATIVE · 0: ZERO · 42: POSITIVE
```

Ne yapıyor? Üç durumdan tam biri gerçekleşir; son durum için koşul yazmaya gerek yok (else). Test ederken her dalı çalıştıran değer seç: negatif, sıfır, pozitif.

Nested if

```java
int age = 25, weight = 48;
if (age >= 18) {
    if (weight > 50) System.out.println("Kan bağışı yapabilirsiniz");
    else System.out.println("Kilo yetersiz (50 kg üstü)");   // çalışır
} else {
    System.out.println("Yaş en az 18 olmalı");
}
```

Ne yapıyor? İçteki if yalnızca dıştaki koşul true ise kontrol edilir. 3+ seviye iç içe if okunabilirliği düşürür; && veya erken return tercih et.

Nested if	**&&**
Her koşula **ayrı mesaj/işlem** gerekiyorsa	**Tek sonuç** varsa daha kısa
if (age >= 18) { if (weight > 50) {...} }	if (age >= 18 && weight > 50) {...}

Ternary alternatifi: `String output = (number % 2 == 0) ? "çift" : "tek";` → basit değer seçiminde ideal; çok satırlı işte if-else.

if hataları

```java
int x = 3;
if (x > 5);            // DİKKAT: boş statement
{ System.out.println("A: Her zaman çalışır"); }     // çalışır
if (x > 5)
    System.out.println("B: koşula bağlı");          // çalışmaz
    System.out.println("C: if'e bağlı DEĞİL");      // çalışır
// if (x = 5) { }  -> compile error: int boolean değil
// if (x) { }      -> compile error (C'den farklı)
```

Ne yapıyor? if (...); → noktalı virgül if'in body'si olur, alttaki block **her zaman** çalışır. { } yoksa **sadece ilk satır** if'e bağlı; girinti anlam taşımaz. Java'da koşul boolean olmalı; 0/1 veya atama kullanılamaz.

switch

```java
switch (expression) {
    case value1:  /* eşleşirse */  break;   // break opsiyonel
    case value2:  /* eşleşirse */  break;
    default:      /* hiçbiri eşleşmezse */  // opsiyonel
}
```

İzin verilen tipler	İzin verilmeyen tipler
byte, short, char, int ve wrapper'ları	**long, float, double, boolean**
String (Java 7+), enum	Kaynakta long deniyor; **yanlış**, compile error

case değerleri	break ve default
Constant olmalı (literal veya final); variable olamaz	break yoksa sonraki case'ler de çalışır (**fall-through**)
Unique olmalı; tekrar → compile error	default opsiyonel, genelde en sonda
Tipi expression ile uyumlu	**Aralık kontrolü yok**: case > 10 yazılamaz

Ay adı örneği:

```java
int month = 7;
String monthName;
switch (month) {
    case 1: monthName = "Ocak"; break;
    // ... 2-12 aynı şekilde
    case 7: monthName = "Temmuz"; break;
    default: monthName = "Geçersiz ay!";
}
System.out.println(month + " - " + monthName);  // 7 - Temmuz
```

Ne yapıyor? Aynı variable'ı birçok sabitle karşılaştırırken uzun if-else-if'ten okunaklı. **default** sayesinde monthName her durumda değer alır; yoksa compile error ("might not have been initialized").

Fall-through

```java
int number = 20;
switch (number) {
    case 10: System.out.println("10");
    case 20: System.out.println("20");   // buradan başlar
    case 30: System.out.println("30");   // break yok → çalışır
    default: System.out.println("Diğer"); // çalışır
}
// Çıktı: 20 / 30 / Diğer
```

Ne yapıyor? Eşleşen case'ten sonra **break'e kadar** tüm satırlar çalışır.

Bilinçli fall-through (case gruplama):

```java
char ch = 'O';
switch (Character.toLowerCase(ch)) {
    case 'a': case 'e': case 'ı': case 'i':
    case 'o': case 'ö': case 'u': case 'ü':
        System.out.println("Vowel (ünlü)");   // çalışır
        break;
    default:
        System.out.println("Consonant");
}
```

Ne yapıyor? Character.toLowerCase harfi küçültür; 8 ünlü aynı koda düşer.

String ve enum ile switch

```java
String levelString = "Expert";
int level;
switch (levelString) {
    case "Beginner": level = 1; break;
    case "Intermediate": level = 2; break;
    case "Expert": level = 3; break;
    default: level = 0;
}
System.out.println("Your Level is: " + level);   // 3
String cmd = "EXIT";
switch (cmd.toLowerCase()) {
    case "exit": System.out.println("Kapanıyor"); break;   // çalışır
    default: System.out.println("Bilinmeyen komut");
}
```

Ne yapıyor? String karşılaştırması **equals()** ile yapılır ve **büyük/küçük harfe duyarlı**; bu yüzden toLowerCase() kullanıldı. Expression **null** ise **NullPointerException**. String switch Java 7 ile geldi.

```java
enum Day { SUN, MON, TUE, WED, THU, FRI, SAT }
for (Day d : Day.values()) {
    switch (d) {
        case SAT:
        case SUN: System.out.println(d + ": Hafta sonu"); break;
        default: System.out.println(d + ": Hafta içi");
    }
}
```

Ne yapıyor? Day.values() tüm sabitleri sırayla döndürür. case'te enum sabiti **ön eksiz** yazılır: case SAT (Day.SAT değil). Wrapper da çalışır: Integer age = 18; switch (age) → unboxing.

Modern switch (Java 14+)

```java
int day = 6;
String type = switch (day) {          // switch expression: değer döndürür
    case 1, 2, 3, 4, 5 -> "Hafta içi";
    case 6, 7 -> "Hafta sonu";
    default -> "Geçersiz";
};
System.out.println(type);             // Hafta sonu
char grade = 'B';
switch (grade) {
    case 'A' -> System.out.println("Mükemmel");
    case 'B', 'C' -> System.out.println("İyi");   // çalışır
    default -> System.out.println("Gelişmeli");
}
```

Ne yapıyor? -> kullanınca **break gerekmez, fall-through olmaz**. Birden fazla label virgülle: case 6, 7. Switch expression değer döndürür; çok satırlı block'ta **yield** kullanılır.

if-else-if mi, switch mi?

Kriter	if-else-if	switch
Koşul türü	Her boolean ifade	Yalnızca eşitlik
Tipler	Tüm tipler	byte, short, char, int (+wrapper), String, enum
Aralık kontrolü	Var: x > 10 && x < 20	Yok
Okunabilirlik	Az koşulda iyi	Çok sabit değerde temiz
Performans	Sırayla kontrol	JVM jump table kullanabilir
Tipik kullanım	Not aralıkları, karmaşık koşul	Menü, gün/ay, komut

PDF 2 · for ve for-each

for anatomisi

```
for (int i = 1; i <= 5; i++) { body }
```

Parça	Ne zaman çalışır
① initialization (int i = 1)	**Bir kez**, en başta
② condition (i <= 5)	**Her turdan önce**; false ise loop biter
③ body	Condition true ise
④ update (i++)	**Her turun sonunda**, sonra ②'ye dönülür

Sıra: ① → ② → ③ → ④ → ② → ③ → ④ → … → ② (false) → çıkış. Tekrar sayısı **biliniyorsa** for tercih edilir.

Temel örnekler

```
for (int i = 1; i <= 10; i++) System.out.print(i + " ");   // 1 2 3 ... 10
System.out.println();
int number = 5, factorial = 1;
for (int i = 1; i <= number; i++) factorial *= i;
System.out.println(number + "! = " + factorial);           // 5! = 120
int sum = 0;
for (int i = 2; i <= 100; i += 2) sum += i;
System.out.println("Çift sayılar toplamı: " + sum);       // 2550
```

Kod	Ne yapıyor?
print / println()	print aynı satıra yazar; println() satır sonu ekler
factorial *= i	factorial = factorial × i; 1×2×3×4×5
i += 2	Update yalnızca i++ olmak zorunda değil; 2'şer artış → sadece çiftler
sum, factorial	**Accumulator**: loop'tan **önce** tanımlanır

Factorial trace:

Tur	i	i <= 5?	factorial önce	factorial sonra
1	1	true	1	1
2	2	true	1	2
3	3	true	2	6
4	4	true	6	24
5	5	true	24	120
—	6	false → biter	120	—

Varyasyonlar

```
for (int i = 10; i >= 1; i -= 3) System.out.print(i + " ");      // 10 7 4 1
for (int i = 0, j = 10; i < j; i++, j--) System.out.print(i + "-" + j + " ");  // 0-10 1-9 2-8 3-7 4-6
int k = 1;
for (; k < 100; ) k *= 3;
System.out.println("k = " + k);                                  // k = 243
for (char c = 'a'; c <= 'e'; c++) System.out.print(c);           // abcde
```

Ne yapıyor? Geriye 3'er sayar; init ve update'te **virgülle** birden fazla ifade yazılabilir; üç kısmın **hepsi opsiyonel** (for (;;) sonsuz loop); loop variable char dahil her tipten olabilir. k: 1 → 3 → 9 → 27 → 81 → 243 (243 < 100 false).

Nested for

```
for (int i = 1; i <= 5; i++) {         // outer: satırlar
    for (int j = 1; j <= 5; j++) {     // inner: sütunlar
        System.out.printf("%4d", i * j);
    }
    System.out.println();
}
// 5x5 çarpım tablosu
```

Ne yapıyor? Inner loop, outer'ın **her turunda baştan sona** çalışır: 5 × 5 = 25 çalışma. printf("%4d", x): sayıyı 4 karakter genişliğinde, sağa hizalı yazar.

Star pattern:

```java
// Left-aligned triangle: i. satırda i yıldız
for (int i = 1; i <= 5; i++) {
    for (int j = 1; j <= i; j++) System.out.print("* ");
    System.out.println();
}
// Inverted triangle: her satırda bir yıldız eksilir
int term = 5;
for (int i = 1; i <= term; i++) {
    for (int j = term; j >= i; j--) System.out.print("* ");
    System.out.println();
}
```

Ne yapıyor? İlkinde inner sınırı **i'ye bağlı** (j <= i) → 1, 2, 3, 4, 5 yıldız. İkincide j term'den i'ye iner → 5, 4, 3, 2, 1 yıldız.

Asal sayı ve Fibonacci

```java
int n = 29;
boolean isPrime = n > 1;
for (int i = 2; i * i <= n; i++) {
    if (n % i == 0) { isPrime = false; break; }
}
System.out.println(n + " asal mı? " + isPrime);   // true

int a = 0, b = 1;
System.out.print("Fibonacci: ");
for (int i = 0; i < 10; i++) {
    System.out.print(a + " ");
    int next = a + b;
    a = b;
    b = next;
}
// 0 1 1 2 3 5 8 13 21 34
```

Kod	Ne yapıyor?
isPrime = n > 1	0 ve 1 asal değil; başlangıç değeri buna göre
i * i <= n	**√n'e kadar** bakmak yeter: büyük bir bölen varsa küçük eşi de vardır
break	Bölen bulununca gereksiz turlar atlanır
next = a + b; a = b; b = next	Her terim önceki ikisinin toplamı; iki variable **kaydırılır**

Labeled break

```java
aa:
for (int i = 1; i <= 3; i++) {
    bb:
    for (int j = 1; j <= 3; j++) {
        if (i == 2 && j == 2) break aa;   // veya break bb;
        System.out.println(i + " " + j);
    }
}
```

Komut	Ne yapıyor?	Çıktı
break aa;	**Outer** loop'u bitirir	1 1, 1 2, 1 3, 2 1
break bb;	Sadece **inner** loop'u bitirir	1 1, 1 2, 1 3, 2 1, 3 1, 3 2, 3 3

for hataları

Hata	Açıklama
Off-by-one	i <= arr.length → son turda ArrayIndexOutOfBoundsException. Doğrusu i < arr.length (index 0'dan başlar)
Loop sonrası ;	for (...); → body boş statement olur, alttaki block yalnızca bir kez çalışır
Yanlış yönde update	for (int i = 0; i < 10; i--) → condition hep true, sonsuz loop
Scope dışı kullanım	for içindeki i loop bitince yok olur; gerekiyorsa loop'tan önce tanımla

for-each (enhanced for)

`for (dataType variable : arrayOrCollection) { ... }` → ":" "içindeki her eleman için" diye okunur. **Java 5** ile geldi; array ve Collection (ArrayList, Set...) üzerinde çalışır. Index yönetimi yok → index-out-of-bounds hatası olmaz.

```java
int[] arr = {12, 23, 44, 56, 78};
for (int x : arr) System.out.print(x + " ");   // 12 23 44 56 78
int total = 0, max = arr[0];
for (int x : arr) {
    total += x;
    if (x > max) max = x;
}
System.out.println("Toplam: " + total);         // 213
System.out.println("Maksimum: " + max);         // 78
double avg = (double) total / arr.length;
System.out.println("Ortalama: " + avg);         // 42.6
```

Ne yapıyor? x her turda sıradaki eleman. max = arr[0] ile ilk elemandan başlar, büyüğünü görünce günceller. (double) cast integer division'ı önler.

```java
ArrayList<String> list = new ArrayList<>();
list.add("Ali"); list.add("Ayşe"); list.add("Mehmet");
for (String name : list) System.out.println(name + " (" + name.length() + ")");
// Ali (3) · Ayşe (4) · Mehmet (6)
int[][] matrix = {{1, 2, 3}, {4, 5, 6}};
for (int[] row : matrix) {          // her satır bir array
    for (int v : row) System.out.print(v + " ");
    System.out.println();
}
```

Ne yapıyor? ArrayList, HashSet gibi tüm **Iterable** yapılarla çalışır (import java.util.ArrayList gerekir). 2D array'de dış loop variable'ı bir **satırdır (int[])**.

Sınırlama:

```java
int[] arr = {1, 2, 3};
for (int x : arr) x = x * 10;                 // ✗ sadece kopya değişir → [1, 2, 3]
for (int i = 0; i < arr.length; i++) arr[i] = arr[i] * 10;  // ✓ [10, 20, 30]
System.out.println(Arrays.toString(arr));
```

Ne yapıyor? Loop variable elemanın **kopyası**; array değişmez. Arrays.toString(arr) array'i [1, 2, 3] biçiminde yazar. for-each sırasında Collection'dan eleman silmek → **ConcurrentModificationException**.

Kriter	Klasik for	for-each
Index erişimi	Var (i)	Yok
Ters / atlayarak gezme	Mümkün	Mümkün değil
Eleman değiştirme	Mümkün (arr[i] = ...)	Mümkün değil
Index hatası riski	Var (off-by-one)	Yok
Okunabilirlik	Orta	Yüksek
Uygun olduğu yer	Sayaç, aralık, index	Tüm elemanları okuma

PDF 2 alıştırma cevapları

S1. `int x = 15; if (x > 10) if (x > 20) System.out.println("A"); else System.out.println("B");`

- Cevap: B
- Neden: else **en yakın if'e** (x > 20) bağlanır. 15 > 10 true, 15 > 20 false → else çalışır.

S2. `int n = 2; switch (n) { case 1: print("1 "); case 2: print("2 "); case 3: print("3 "); }`

- Cevap: 2 3
- Neden: case 2 eşleşir, break olmadığı için case 3'e de düşer (fall-through).

S3. `int count = 0; for (int i = 0; i < 5; i++) for (int j = i; j < 5; j++) count++;`

- Cevap: 15
- Neden: j, i'den başlar. i = 0 → 5 tur, 1 → 4, 2 → 3, 3 → 2, 4 → 1 → 5 + 4 + 3 + 2 + 1 = 15.

S4. `int s = 0; for (int v : new int[]{3, 1, 4, 1, 5}) if (v % 2 == 1) s += v;`

- Cevap: 10
- Neden: Sadece tekler toplanır: 3 + 1 + 1 + 5 = 10.

PDF 3 · while, do-while, break, continue, comments

while

```
while (condition) {
    // body
    // update (i++)
}
```

- Condition true olduğu sürece body tekrar çalışır.
- **Entry-controlled**: condition her turdan **önce** kontrol edilir → body **hiç çalışmayabilir**.
- Initialization loop'tan önce, update body içinde yapılır; **update unutulursa sonsuz loop**.
- Tekrar sayısı **bilinmiyorsa** tercih edilir: kullanıcı girdisi, dosya okuma, oyun döngüsü.

```
int i = 1;                          // initialization
while (i <= 5) {                    // condition
    System.out.println("i = " + i); // i = 1 ... i = 5
    i++;                            // update
}
System.out.println("Loop bitti, i = " + i);  // Loop bitti, i = 6
int x = 100;
while (x < 10) { System.out.println("Bu satır hiç yazılmaz"); }
```

Ne yapıyor? Loop bitince i = 6: condition'ı false yapan **ilk** değer. İkinci loop'ta condition baştan false → body **sıfır kez** çalışır.

Basamak işlemleri

```
int number = 7351;
int sum = 0, reversed = 0, digits = 0;
int n = number;                     // orijinali korumak için kopya
while (n > 0) {
    int digit = n % 10;             // son basamak
    sum += digit;
    reversed = reversed * 10 + digit;
    digits++;
    n /= 10;                        // son basamağı at
}
// Basamak sayısı: 4 · Basamaklar toplamı: 16 · Ters çevrilmiş: 1537
```

Kod	Ne yapıyor?
n % 10	Son basamağı verir
n /= 10	Son basamağı atar
reversed * 10 + digit	Mevcut sonucu bir basamak sola kaydırıp yeni basamağı ekler
int n = number	Orijinal değer bozulmasın diye kopya üzerinde çalışılır

Tur sayısı basamak sayısına bağlı, önceden bilinmediği için while uygun.

Tur	n (başta)	digit	sum	reversed	n /= 10
1	7351	1	1	1	735
2	735	5	6	15	73
3	73	3	9	153	7
4	7	7	16	1537	0
—	0	n > 0 false → biter			

Slayt alıştırması — number = 4096:

Tur	n	digit	sum	reversed	n /= 10
1	4096	6	6	6	409
2	409	9	15	69	40
3	40	0	15	690	4
4	4	4	19	6904	0

Sonuç **6904**. Sıfır kaybolmaz çünkü baştaki değil **ortadaki** basamak: 69 × 10 + 0 = 690 ile korunur, sonra 4 eklenir. Kaybolan sıfır yalnızca sondaki sıfırlardır (ör. 4090 → 904).

EBOB (Euclid) ve Collatz

```java
int a = 48, b = 18;
while (b != 0) {          // Euclid
    int temp = b;
    b = a % b;
    a = temp;
}
System.out.println("EBOB(48, 18) = " + a);   // 6

int n = 6, steps = 0;
System.out.print(n);
while (n != 1) {          // Collatz
    n = (n % 2 == 0) ? n / 2 : 3 * n + 1;
    System.out.print(" " + n);
    steps++;
}
System.out.println("\nAdım sayısı: " + steps);
// 6 3 10 5 16 8 4 2 1 · Adım sayısı: 8
```

Ne yapıyor? Euclid: EBOB(a, b) = EBOB(b, a % b); b sıfır olunca a sonuçtur (48,18 → 18,12 → 12,6 → 6,0). Collatz: çiftse ikiye böl, tekse 3n+1; kaç adımda 1'e ulaşacağı bilinmez. İki örnekte de koşul **sayaçtan değil veriden** gelir.

Nested while

```java
int i = 1;
while (i <= 5) {
    int j = 1;                    // her satırda sıfırla!
    while (j <= 5) {
        System.out.printf("%4d", i * j);
        j++;
    }
    System.out.println();
    i++;
}
```

Ne yapıyor? 5x5 çarpım tablosu. Inner sayaç (j) **outer'ın içinde** sıfırlanmalı; dışarı alınırsa inner loop yalnızca ilk satırda çalışır. Sayaçlı nested yapıda nested for daha okunaklı.

Sonsuz loop

```java
int count = 0;
while (count < 5) {
    System.out.println("Count: " + count);
    // count++ unutuldu! → Count: 0 sonsuza kadar
}
```

Ne yapıyor? Condition hiç false olmaz. Konsolda durdurmak: **Ctrl+C**. Doğrusu count++ eklemek → Count: 0 … 4. Bilinçli sonsuz loop: **while (true) + içeride break** (sunucular, oyun döngüleri).

Sentinel-controlled loop

```java
Scanner sc = new Scanner(System.in);
int sum = 0, count = 0;
System.out.print("Sayı girin (-1 = bitir): ");
int n = sc.nextInt();
while (n != -1) {                 // sentinel kontrolü
    sum += n;
    count++;
    System.out.print("Sayı girin (-1 = bitir): ");
    n = sc.nextInt();
}
if (count > 0) System.out.println("Ortalama: " + (double) sum / count);
else System.out.println("Hiç sayı girilmedi");
sc.close();
// Girdi 10, 20, 45, -1 → Ortalama: 25.0
```

Kod	Ne yapıyor?
new Scanner(System.in)	Klavyeden okuma; import java.util.Scanner gerekir
sc.nextInt()	Bir tamsayı okur
n != -1	**Sentinel**: girişin bittiğini gösteren özel değer
count > 0	0/0 bölmesini (**NaN**) önler
(double) sum / count	Integer division'ı önler
sc.close()	Scanner'ı kapatır

do-while

```
do {
    // body
    // update
} while (condition);    // ; zorunlu!
```

- **Exit-controlled**: condition her turun **sonunda** kontrol edilir.
- Body **en az bir kez** çalışır; condition baştan false olsa bile.
- while (condition) satırının sonundaki **noktalı virgül zorunlu**.
- Tipik kullanım: menü, geçerli girdi alana kadar tekrar sorma.

```
int i = 10;
while (i < 5) { System.out.println("while: " + i); i++; }
System.out.println("while bitti");          // sadece: while bitti

int k = 10;
do { System.out.println("do-while: " + k); k++; } while (k < 5);
System.out.println("do-while bitti");       // do-while: 10, do-while bitti
```

Ne yapıyor? while'da 10 < 5 false → body hiç çalışmaz. do-while'da body **bir kez** çalışır, sonra kontrol edilir.

```
int i = 1;
do { System.out.print(i + " "); i++; } while (i <= 10);   // 1 2 ... 10
int number = 5, factorial = 1, k = 1;
do { factorial *= k; k++; } while (k <= number);
System.out.println(number + "! = " + factorial);          // 5! = 120
```

Ne yapıyor? Aynı işler for/while ile de yapılabilir; seçim okunabilirliğe göre. Body'nin en az bir kez çalışması gerekmiyorsa while/for daha doğal.

Input validation:

```
Scanner sc = new Scanner(System.in);
int grade;
do {
    System.out.print("Not girin (0-100): ");
    grade = sc.nextInt();
    if (grade < 0 || grade > 100) System.out.println("Geçersiz! Tekrar deneyin.");
} while (grade < 0 || grade > 100);
System.out.println("Kaydedilen not: " + grade);
// 150 → Geçersiz, -5 → Geçersiz, 85 → Kaydedilen not: 85
```

Ne yapıyor? Kullanıcıya en az bir kez sormak gerektiği için do-while en doğal seçim; geçerli değer girilene kadar tekrar sorar.

Menü:

```
int choice;
do {
    System.out.println("1) Selamla 2) Yardım 0) Çıkış");
    System.out.print("Seçiminiz: ");
    choice = sc.nextInt();
    switch (choice) {
        case 1 -> System.out.println("Merhaba!");
        case 2 -> System.out.println("Bir numara girin.");
        case 0 -> System.out.println("Güle güle");
        default -> System.out.println("Geçersiz seçim");
    }
} while (choice != 0);
```

Ne yapıyor? Menü en az bir kez gösterilmeli → do-while. 0 seçilince condition false olur, program biter.

for, while, do-while karşılaştırması

Kriter	for	while	do-while
Condition kontrolü	Her turdan önce	Her turdan önce	Her turdan **sonra**
Loop türü	Entry-controlled	Entry-controlled	**Exit-controlled**
Minimum çalışma	0	0	**1**
Ne zaman?	Tekrar sayısı biliniyor	Bilinmiyor	Bilinmiyor ve en az 1 kez gerekli
Sonsuz loop	for (;;) { }	while (true) { }	do { } while (true);
Tipik örnek	1'den 100'e toplam	Dosya sonuna kadar okuma	Menü, girdi doğrulama

break

İçinde bulunduğu loop'u veya switch'i **anında ve tamamen** sonlandırır; kontrol loop'tan sonraki ilk satıra geçer. for, while, do-while ve switch dışında kullanılırsa **compile error**.

```
int[] array = {7, 2, 9, 4, 5};
int target = 9;
int foundIndex = -1;                    // -1: bulunamadı
for (int i = 0; i < array.length; i++) {
    System.out.println("Kontrol: index " + i);
    if (array[i] == target) { foundIndex = i; break; }
}
System.out.println("Bulunan index: " + foundIndex);
// Kontrol: index 0, 1, 2 · Bulunan index: 2
```

Ne yapıyor? Hedef bulununca aramayı bitirir; index 3 ve 4 hiç kontrol edilmez → gereksiz iş yapılmaz.

```
int i = 1;
while (true) {
    if (i * i > 50) break;
    System.out.print(i * i + " ");
    i++;
}
System.out.println("\nSonuç: " + i * i);   // 1 4 9 16 25 36 49 · Sonuç: 64

int k = 1;
do {
    if (k == 5) break;                      // condition hâlâ true olsa bile
    System.out.print(k + " ");
    k++;
} while (k <= 10);                          // 1 2 3 4
```

Ne yapıyor? while (true) + break: çıkış koşulu loop'un **ortasında** olduğunda kullanışlı. 8 × 8 = 64 > 50 → çıkar. do-while'da break, condition true olsa bile loop'u bitirir.

Labeled break: nested loop'tan tamamen çıkmak

```
int[][] matrix = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
int target = 5;
boolean found = false;
search:                                     // label
for (int r = 0; r < matrix.length; r++) {
    for (int c = 0; c < matrix[r].length; c++) {
        if (matrix[r][c] == target) {
            System.out.println("Bulundu: [" + r + "][" + c + "]");  // [1][1]
            found = true;
            break search;                   // iki loop'tan da çık
        }
    }
}
System.out.println("found = " + found);    // true
```

Ne yapıyor? Etiketsiz break yalnızca **en içteki** loop'u bitirir. Label, loop'un önüne **isim:** şeklinde yazılır. Labeled break Java'nın ilk sürümünden beri var.

Loop içindeki switch'te break:

```
for (int i = 1; i <= 3; i++) {
    switch (i) {
        case 2: System.out.println("2 bulundu"); break;   // sadece switch'ten çıkar
        default: System.out.println(i);
    }
}
// 1 · 2 bulundu · 3  (loop devam eder)
```

Ne yapıyor? switch içindeki break **sadece switch'ten** çıkar. Loop'tan çıkmak için **labeled break** (loop: ... break loop; → çıktı:

1, 2 bulundu), boolean flag veya method'dan return.

continue

Mevcut turun **kalanını atlar**; loop bitmez.

```
for (int i = 1; i <= 10; i++) {
    if (i == 5) continue;                  // 5'i atla
    System.out.print(i + " ");             // 1 2 3 4 6 7 8 9 10
}
for (int i = 1; i <= 10; i++) {
    if (i % 2 == 0) continue;              // çiftleri atla
    System.out.print(i + " ");             // 1 3 5 7 9
}
```

Ne yapıyor? for loop'ta continue'dan sonra **update (i++) yine çalışır**. while/do-while'da doğrudan condition kontrolüne geçilir.

Nested ve labeled continue:

```
aa:
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        if (i == 2 && j == 2) continue;      // veya continue aa;
        System.out.println(i + " " + j);
    }
}
```

Komut	Ne yapıyor?	Atlanan
continue;	Inner loop'un sonraki turu	Sadece 2 2
continue aa;	**Outer** loop'un sonraki turu	2 2 ve 2 3 (i = 2 satırının kalanı)

while içinde continue tuzağı:

```
int i = 1;
while (i <= 10) {
    if (i == 5) continue;     // ✗ i++ atlanır → i hep 5, sonsuz loop (çıktı: 1 2 3 4)
    System.out.println(i);
    i++;
}
// ✓ Doğrusu: if (i == 5) { i++; continue; } → 1 2 3 4 6 7 8 9 10
```

Guard clause (veri filtreleme):

```
String[] names = {"Ali", null, "Ayşe", "", "Mehmet", null};
int printed = 0;
for (String name : names) {
    if (name == null) continue;       // null'ları atla
    if (name.isEmpty()) continue;     // boşları atla
    System.out.println(name);
    printed++;
}
System.out.println(printed + " isim yazdırıldı");  // Ali · Ayşe · Mehmet · 3 isim yazdırıldı
```

Ne yapıyor? Guard clause: istenmeyen durumları turun başında eleyip ana işi sade bırakmak; iç içe if'leri azaltır. Çok sayıda continue takibi zorlaştırır; tek koşul daha iyi olabilir: if (name != null && !name.isEmpty()). null kontrolü önce yapılmalı, yoksa name.isEmpty() NullPointerException verir.

Jump statements özeti

	break	continue	return
Ne yapar	Loop'u veya switch'i **tamamen** bitirir	Yalnızca **mevcut turu** atlar	İçinde bulunduğu **method'u** bitirir
Nerede	Loop + switch	**Sadece loop** (switch tek başına değil)	Method
Ek	Labeled hali dıştaki loop'u bitirir	Labeled hali dış loop'un sonraki turuna geçer	Değer döndürebilir: return a + b; main içinde programı bitirir
Örnek	Aranan eleman bulununca dur	Geçersiz veriyi atla	Sonuç döndür

Comments

Compiler ve JVM tarafından **yok sayılır**. Amaç: okunabilirlik, bakım kolaylığı, kodu silmeden geçici devre dışı bırakma.

Tür	Söz dizimi	Kullanım
Single-line	// satır sonuna kadar	Kısa açıklama; en sık kullanılan
Multi-line	/* ... */	Uzun açıklama, kod bloğu kapatma; **iç içe yazılamaz**
Documentation	/** ... */	**javadoc** aracıyla HTML API dokümanı üretir; @param, @return gibi tag'ler; public class/method için

```
int i = 10;                 // i, değeri 10 olan bir variable
/* Aşağıdaki kod geçici olarak devre dışı:
float j = 5.9f;
System.out.println(j);
*/
System.out.println(i);      // 10
// System.out.println("test");  -> çalışmaz
```

Ne yapıyor? `/* */` arasındaki kod derlenmez. `/* /* … */ */` iç içe → **compile error**. IDE kısayolu: seçili satırları yorum yapmak için **Ctrl + /**.

Javadoc:

```
/**
 * Basit aritmetik işlemler yapan sınıf.
 * @author Ad Soyad
 * @version 1.0
 */
public class Calculate {
    /**
     * İki tamsayının toplamını hesaplar.
     * @param a birinci sayı
     * @param b ikinci sayı
     * @return a ile b'nin toplamı
     */
    public int sum(int a, int b) { return a + b; }
    public static void main(String[] args) {
        System.out.println(new Calculate().sum(40, 20));   // 60
    }
}
```

Ne yapıyor? /** ile başlar, method/class tanımının **hemen üstüne** yazılır. HTML doküman üretmek için: `javadoc -d docs Calculate.java`. IDE'ler Javadoc'u kod tamamlama ve hover penceresinde gösterir.

Tag	Örnek	Açıklama
@author	@author Ad Soyad	Class'ın yazarı
@version	@version 1.0	Sürüm
@since	@since 2026-11-01	Hangi sürümden/tarihten beri var
@param	@param a birinci sayı	Parametre açıklaması
@return	@return toplam	Dönüş değeri (void hariç)
@throws	@throws IOException açıklama	Fırlatılabilecek exception
@see	@see String#length()	İlgili class/method'a referans
{@code}	{@code a + b}	Metni kod fontuyla, HTML yorumlamadan gösterir
@deprecated	@deprecated yerine x() kullanın	Artık kullanılmaması gereken öğe

Java 23+ ile /// ile başlayan **Markdown documentation comment** de desteklenir (JEP 467).

İyi comment / kötü comment:

✗ Kötü: kodu tekrar eder	✓ İyi: NEDEN'i açıklar
int x = 5; // x'e 5 ata	// Sunucu geçici hata verebilir; 5 denemeden sonra vazgeçiyoruz. final int MAX_RETRY = 5;
x++; // x'i 1 artır	// TODO: deneme aralığını ayarlanabilir yap

Kural: İyi kod **ne'yi**, iyi comment **neden'i** anlatır. Anlamsız isim + comment yerine anlamlı isim kullan. **TODO, FIXME** etiketleri IDE'lerde listelenir. Kod değişince comment'i de güncelle.

Çalışan comment (merak konusu):

```
// \u000d System.out.println("Comment çalıştı!");
// Çıktı: Comment çalıştı!
```

Ne yapıyor? Java, Unicode escape'leri (\uXXXX) lexical analizden **önce** karaktere çevirir. \u000d satır sonu karakteridir: println yeni satıra düşer ve comment dışında kalır. Gerçek kodda asla kullanma.

PDF 3 alıştırma cevapları

S1. `int i = 10, c = 0; while (i > 0) { i -= 3; c++; } System.out.println(i + " " + c);`

- Cevap: -2 4
- Neden: i: 10 → 7 → 4 → 1 → −2; dört tur. −2 > 0 false → biter.

S2. `int n = 0; do { n++; } while (n > 5); System.out.println(n);`

- Cevap: 1
- Neden: do-while body bir kez çalışır (n = 1), sonra 1 > 5 false → biter.

S3. `for (int i = 1; i <= 10; i++) { if (i % 3 == 0) continue; if (i > 7) break; System.out.print(i + " "); }`

- Cevap: 1 2 4 5 7
- Neden: 3 ve 6 continue ile atlanır. 8'de i > 7 → break. (9 zaten 3'e bölünür ama oraya gelinmez.)

S4. `int x = 5; /* x = x * 2; */ x += 1; // x = 100;`

- Cevap: 6
- Neden: İki atama da comment içinde, çalışmaz. Sadece x += 1 → 6.

PDF 4 · OOP kavramları ve naming convention

Procedural vs object-oriented

Kriter	Procedure-oriented (C)	Object-oriented (Java)
Temel birim	Fonksiyon	Object ve class
Veri ve davranış	Ayrı: veri fonksiyonlara parametre olarak gider	**Birlikte**: veri ve method aynı class'ta
Veri erişimi	Global veriye her yerden	**Data hiding**: private alanlar korunur
Büyük projede bakım	Kod büyüdükçe zorlaşır	Modüler; bakım kolay
Yeniden kullanım	Kopyala-yapıştır	Inheritance ve composition
Gerçek dünya modellemesi	Dolaylı	Doğrudan: Öğrenci, Hesap, Araba

Tarihçe: Simula (1967) ilk object-oriented dil; **Smalltalk** her şeyin object olduğu ilk "saf" OOP dili. Bugün Java, C#, C++, Python, PHP yaygın.

6 temel kavram

Kavram	Tanım
Object	State (durum) ve behavior (davranış) sahibi varlık: kalem, araba, banka hesabı
Class	Object'lerin **şablonu (blueprint)**; hangi field ve method olacağını tanımlar
Inheritance	Bir class'ın başka class'ın özelliklerini devralması (**IS-A**); kod tekrarını azaltır
Polymorphism	Aynı işin farklı biçimde yapılması: **overloading (compile-time)** ve **overriding (runtime)**
Abstraction	İç detayı gizleyip sadece işlevi göstermek: **abstract class ve interface**
Encapsulation	Veri ve kodu tek birimde toplamak; veriyi **private** yapıp **getter/setter** ile korumak

Object: state, behavior, identity

Bileşen	Karşılığı	Örnek (myDog)
State	Field'lar	name = "Rocky", breed = "Golden", age = 3
Behavior	Method'lar	bark(), eat(), wagTail()
Identity	JVM'deki benzersiz kimlik	Aynı değerli iki object yine **farklı** object

Object fiziksel (araba) veya mantıksal (banka hesabı, sipariş) olabilir. Object bir class'ın **instance**'ıdır ve **heap**'te yer kaplar.

```
class Dog {
    String name;                  // state (field)
    void bark() {                 // behavior (method)
        System.out.println(name + " says Woof!");
    }
}
Dog myDog = new Dog();            // object oluştur
myDog.name = "Rocky";             // field'a değer ata
myDog.bark();                     // Rocky says Woof!
Dog other = new Dog();
other.name = "Karabaş";
other.bark();                     // Karabaş says Woof!
```

Ne yapıyor? Dog bir class; myDog ve other ondan üretilen **iki ayrı object**. new Dog() heap'te yeni object oluşturur. **Nokta (.)** ile field ve method'a erişilir.

Class tanımının parçaları

`public class Car extends Vehicle implements Drivable { … }`

Parça	Açıklama
public	Modifier: class **public** veya **default** (package-private) olabilir
class Car	Keyword + ad; ad **PascalCase**
extends Vehicle	**Yalnızca bir** parent class (Java'da çoklu class inheritance yok)
implements Drivable	Virgülle **birden fazla** interface
{ ... }	Body: field, method, constructor, block, nested class

Inheritance

```
class Vehicle {                               // superclass
    void start() { System.out.println("Vehicle is starting..."); }
    void stop() { System.out.println("Vehicle is stopping..."); }
}
class Car extends Vehicle {                   // subclass
    void honk() { System.out.println("Car is honking!"); }
}
Car myCar = new Car();
myCar.start();   // Vehicle'dan miras
myCar.stop();    // Vehicle'dan miras
myCar.honk();    // kendi method'u
```

Ne yapıyor? "Car bir Vehicle'dır" → IS-A. Car, start() ve stop()'u yeniden yazmadan kullanır → **code reusability**. Inheritance, runtime polymorphism'in temelidir.

Polymorphism

```
class Animal { void sound() { System.out.println("..."); } }
class Dog extends Animal { @Override void sound() { System.out.println("Hav hav"); } }
class Cat extends Animal { @Override void sound() { System.out.println("Miyav"); } }
Animal[] animals = { new Dog(), new Cat(), new Animal() };
for (Animal a : animals) a.sound();   // Hav hav · Miyav · ...
```

Ne yapıyor? Aynı a.sound() çağrısı, object'in **gerçek tipine** göre farklı çalışır; karar **runtime'da** verilir. Runtime polymorphism = overriding; compile-time polymorphism = overloading (aynı isim, farklı parametre).

Abstraction

```
abstract class Animal {
    abstract void makeSound();               // gövdesiz: "ne" yapılacak
    void breathe() { System.out.println("Animal is breathing..."); }  // concrete
}
class Dog extends Animal {
    @Override void makeSound() { System.out.println("Dog barking"); } // "nasıl"
}
Animal myDog = new Dog();
myDog.breathe();     // Animal is breathing...
myDog.makeSound();   // Dog barking
// Animal a = new Animal(); -> HATA: abstract
```

Ne yapıyor? Kullanıcı **ne** yapıldığını bilir, **nasıl** yapıldığını bilmesi gerekmez (telefonla arama örneği). abstract method gövdesizdir, subclass doldurur. **abstract class'tan doğrudan object oluşturulamaz.** Java'da abstraction: abstract class ve interface.

Encapsulation

```
class BankAccount {
    private double balance;                      // dışarıdan erişilemez
    public void deposit(double amount) {
        if (amount > 0) balance += amount;
        else System.out.println("Geçersiz miktar!");
    }
    public double getBalance() { return balance; } // getter
}
BankAccount acc = new BankAccount();
acc.deposit(500);
acc.deposit(-100);                               // Geçersiz miktar!
// acc.balance = 1000000; -> HATA: private
System.out.println("Bakiye: " + acc.getBalance()); // Bakiye: 500.0
```

Ne yapıyor? Veri private, erişim kontrollü public method'larla. deposit içinde **doğrulama** yapılarak geçersiz durum engellenir. Tüm field'ları private olan class'a **JavaBean** denir.

Coupling ve cohesion

Coupling (bağımlılık)	Cohesion (bütünlük)
Bir class'ın başka class'a ne kadar bağımlı olduğu	Bir class/method'un tek ve iyi tanımlı işe ne kadar odaklandığı
Tight: class diğerini doğrudan oluşturur, detaylarına bağlı	High: Calculator yalnızca hesap yapar
Loose hedeflenir: interface ve access modifier'lar azaltır	java.io paketi yüksek cohesion örneği (hepsi I/O)
Hedef: düşük coupling	**Hedef: yüksek cohesion**

Nesneler arası ilişkiler

İlişki	Anlam	Örnek	Bağın gücü
Inheritance (IS-A)	Bir class diğerinin türü	Car IS-A Vehicle	Class düzeyi
Association	Object'ler birbirini kullanır/bilir	Ülke – Başbakan (one-to-one)	Genel
Aggregation (HAS-A)	Parça **bağımsız** yaşayabilir	Üniversite – Bölüm	**Zayıf**
Composition (HAS-A)	Parça bütün olmadan **anlamsız**	Araba – Motor	**Güçlü**

Association türleri: one-to-one, one-to-many, many-to-one, many-to-many; tek yönlü (unidirectional) veya çift yönlü (bidirectional). Aggregation ve composition, association'ın özel hâlleri.

```
// Aggregation: parça dışarıdan gelir
class Department { String name; Department(String n) { name = n; } }
class University {
    Department dept;                        // HAS-A
    University(Department d) { dept = d; } // dışarıdan
}
// Composition: parça içeride oluşur
class Engine { void start() { System.out.println("Motor çalıştı"); } }
class Car {
    private final Engine engine = new Engine();   // içeride
    void drive() { engine.start(); }
}
```

Ne yapıyor? Aggregation'da Department önce oluşturulup University'ye verilir; University yok olsa da başka yerde kullanılabilir. Composition'da motor araba ile birlikte oluşur ve onunla yaşar. Java'da object'ler "silinmez"; erişilemez hâle gelince **garbage collector** temizler.

Naming convention

Identifier	Kural	Örnek
Class	Büyük harfle, **isim (noun)**; PascalCase	Employee, BankAccount, String
Interface	Büyük harfle, genelde **sıfat (adjective)**	Runnable, Comparable, Printable
Method	Küçük harfle, **fiil (verb)**; camelCase	print(), calculateArea(), getName()
Variable	Küçük harfle; camelCase; _ veya $ ile başlamaz	firstName, totalPrice, id
Constant	Tamamı büyük; kelimeler _ ile	MAX_SIZE, MIN_AGE, PI
Package	Tamamı küçük; ters domain	java.util, tr.edu.gumushane.proje

Bunlar **convention**: compiler zorlamaz, ama endüstri standardı.

System.out.println() ne söyler? System = class (PascalCase) · out = static field (camelCase) · println = method (camelCase, fiil).

✗ Convention'a uymayan	✓ Uygun
class student_record	class StudentRecord
int X;	int studentId;
static final int maxsize = 50;	static final int MAX_SIZE = 50;
void Print_Info()	void printInfo()

Kötü hâli **derlenir** ama okuyanı yanıltır (X ne? maxsize sabit mi?). Tek harfli isim (x, y) yalnızca loop sayacı gibi kısa ömürlü yerlerde.

PDF 4 · Class & Object, Methods, Constructors

Class = blueprint, object = instance

Class bir **şablondur**, bellekte object yoktur. new Car() ile üretilen car1, car2, car3 heap'te **kendi değerleriyle** yaşar (Kırmızı/120, Mavi/90, Beyaz/0).

Class içerebilir	Açıklama
Fields	Object'in durumunu tutan variable'lar (instance variable); private/public alabilir
Methods	Davranışlar; değer döndürür veya void
Constructors	new ile object oluşurken çalışır, ilk değerleri verir
Blocks	**Instance block**: her object oluşturulurken çalışır. **Static block**: class yüklenirken bir kez
Nested class / interface	Class içinde tanımlanan yapılar; mantıksal gruplama

Field default değerleri

```
class Student {
    int id;
    String name;
    double gpa;
    boolean active;
}
Student s1 = new Student();
System.out.println(s1.id);      // 0
System.out.println(s1.name);    // null
System.out.println(s1.gpa);     // 0.0
System.out.println(s1.active);  // false
```

Ne yapıyor? Değer verilmeyen field: sayılar **0**, boolean **false**, reference **null**. Local variable default almaz. Gerçek projede her class ayrı .java dosyasında, main ayrı class'ta.

Object initialization: 3 yol

```java
class Student {
    int rollno;
    String name;
    void insertRecord(int r, String n) { rollno = r; name = n; }   // method ile
    void displayInformation() { System.out.println(rollno + " " + name); }
}
Student s1 = new Student();
s1.rollno = 101;                  // 1) reference ile
s1.name = "Ali";
Student s2 = new Student();
s2.insertRecord(222, "Ayşe");     // 2) method ile
s1.displayInformation();          // 101 Ali
s2.displayInformation();          // 222 Ayşe
```

Ne yapıyor? Üç yol: **reference variable, method, constructor**. Field'lara doğrudan atama encapsulation'ı bozar; method/constructor tercih edilir.

Reference variable

```java
class Point { int x, y; }
Point p1 = new Point();
p1.x = 5;
Point p2 = p1;                    // YENİ object yok, aynı adres!
p2.x = 99;
System.out.println(p1.x);         // 99
Point p3 = new Point();
p3.x = 99;
System.out.println(p1 == p2);     // true  aynı object
System.out.println(p1 == p3);     // false farklı object
```

Ne yapıyor? p1, p2, p3 object'in kendisi değil, object'e işaret eden **reference**. p2 = p1 ile iki reference aynı object'i gösterir (**aliasing**); p2 üzerinden değişiklik p1'de görünür. == reference karşılaştırır: içerik aynı olsa da farklı object için false.

Banka hesabı örneği

```java
class Account {
    int accNo;
    String name;
    double amount;
    void insert(int a, String n, double amt) { accNo = a; name = n; amount = amt; }
    void deposit(double amt) { amount += amt; System.out.println(amt + " yatırıldı"); }
    void withdraw(double amt) {
        if (amount < amt) System.out.println("Yetersiz bakiye");
        else { amount -= amt; System.out.println(amt + " çekildi"); }
    }
    void checkBalance() { System.out.println("Bakiye: " + amount); }
}
// main: a1.insert(832345, "Ankit", 1000); checkBalance(); deposit(40000);
//       withdraw(15000); withdraw(50000); checkBalance();
// Bakiye: 1000.0 · 40000.0 yatırıldı · 15000.0 çekildi · Yetersiz bakiye · Bakiye: 26000.0
```

Method	Ne yapıyor?
insert	Hesap no, isim, bakiye atar
deposit	Bakiyeye ekler
withdraw	Bakiye yetmiyorsa uyarır, yetiyorsa düşer (26000 < 50000 → yetersiz)
checkBalance	Bakiyeyi yazar

Her method object'in **kendi** amount değerini değiştirir: 1000 + 40000 − 15000 = 26000.

Anonymous object

```java
class Calculation {
    void fact(int n) {
        int fact = 1;
        for (int i = 1; i <= n; i++) fact *= i;
        System.out.println("factorial is " + fact);
    }
}
Calculation c = new Calculation();                    // referanslı
c.fact(4);                                            // factorial is 24
new Calculation().fact(5);                            // anonymous → 120
Calculation c1 = new Calculation(), c2 = new Calculation();
c1.fact(3);                                           // factorial is 6
```

Ne yapıyor? Anonymous object: reference'a atanmayan, bir kez kullanılan object; sonra erişilemez, garbage collector toplar. Tek satırda virgülle birden fazla object oluşturulabilir.

Object oluşturmanın yolları

Yol	Açıklama
new keyword	En yaygın; bellek ayırır ve constructor'ı çağırır
Reflection	Class çalışma zamanında bulunur, constructor çağrılır; eski newInstance() deprecated
clone()	Var olan object'in kopyası; varsayılan **shallow copy**; class **Cloneable** olmalı
Deserialization	Dosya/ağdan gelen byte'lardan object; ObjectInputStream
Factory method	Object döndüren static method: LocalDate.now()

Class vs object

Özellik	Class	Object
Tanım	Şablon	Class'ın instance'ı
Tür	Mantıksal varlık	Çalışma zamanında var olan
Bellek	JVM'e yüklenince bir kez (Method Area)	Her object için heap'te ayrı
Oluşturma	class ile bir kez	new ile istenildiği kadar
İçerik	Field ve method tanımları	Field'ların gerçek değerleri
Ömür	Class yüklü kaldıkça	Reference'ı oldukça (sonra GC)
Inheritance	extends ile genişletilir	Object'ten türetilemez

Method anatomisi

```
public static int add(int a, int b) { return a + b; }
```

Parça	Açıklama
public	Access modifier
static	Opsiyonel
int	Return type; değer döndürmüyorsa **void**
add	Method adı: **fiil, camelCase**
(int a, int b)	Parameter list; yoksa ()
{ return a + b; }	Method body

Method signature = ad + parametre listesi (return type dahil değil). Method yalnızca çağrıldığında çalışır. Java'da her method bir **class içinde** olmak zorunda.

Faydalar: Reusability (bir kez yaz, çok yerden çağır) · Modularity (problemi parçala) · Maintainability (değişiklik tek yerde) · Abstraction (içini bilmeden kullan: Math.sqrt(16)).

Access modifier'lar

Modifier	Aynı class	Aynı package	Farklı package'taki subclass	Her yer
public	✓	✓	✓	✓
protected	✓	✓	✓	✗
default (yazılmazsa)	✓	✓	✗	✗
private	✓	✗	✗	✗

Genel kural: field'lar private, dışarıya sunulan method'lar public. En kısıtlayıcı uygun erişim = encapsulation'ın temeli.

Built-in method'lar

```
System.out.println(Math.pow(2, 5));     // 32.0
System.out.println(Math.sqrt(144));     // 12.0
System.out.println(Math.max(17, 42));   // 42
System.out.println(Math.abs(-7.5));     // 7.5
System.out.println(Math.round(4.6));    // 5
String s = "Java";
System.out.println(s.length());         // 4
System.out.println(s.charAt(0));        // J
System.out.println(s.toUpperCase());    // JAVA
System.out.println(s.equals("java"));   // false
System.out.println(Integer.parseInt("25") + 5);  // 30
```

Method	Ne yapıyor?
Math.pow(2, 5)	2^5; sonuç **double**
Math.sqrt(144)	Karekök
Math.max / abs / round	En büyük / mutlak değer / yuvarlama
s.charAt(0)	0. indeksteki karakter
s.equals("java")	Büyük/küçük harf duyarlı → false

Math method'ları **static**: class adıyla çağrılır. String method'ları ise object üzerinden çağrılır.

Static ve instance method

```
public class Main {
    static int square(int x) { return x * x; }            // static
    void greet(String name) { System.out.println("Merhaba " + name); }  // instance
    public static void main(String[] args) {
        System.out.println(square(6));        // 36  aynı class: doğrudan
        System.out.println(Main.square(3));   // 9   class adıyla
        Main obj = new Main();
        obj.greet("Ayşe");                    // Merhaba Ayşe
        // greet("Ali"); -> HATA: static context'ten çağrılamaz
    }
}
```

Ne yapıyor? static method object **gerektirmez**: ClassName.method(). Instance method object üzerinden: obj.method(). **static method'dan instance method'u object olmadan çağırmak → compile error.**

Parametreler ve return

```
static boolean isEven(int n) { return n % 2 == 0; }
static double average(int[] nums) {
    int sum = 0;
    for (int v : nums) sum += v;
    return (double) sum / nums.length;
}
static void printLine(int len) {
    for (int i = 0; i < len; i++) System.out.print("-");
    System.out.println();
}
System.out.println(isEven(10));                    // true
System.out.println(average(new int[]{3, 4, 8}));  // 5.0
printLine(12);                                     // ------------
```

Ne yapıyor? return değer döndürür ve method'u **hemen** bitirir. void olmayan method'un **her yolu** değer döndürmeli. Çağıranın verdiği değer **argument**, method tanımındaki **parameter**.

Java her zaman pass-by-value

```java
static void change(int x) { x = 100; }          // kopya değişir
static void changeArray(int[] a) { a[0] = 100; } // aynı array değişir
int n = 5;
change(n);
System.out.println(n);        // 5
int[] arr = {5, 6};
changeArray(arr);
System.out.println(arr[0]);   // 100
```

Ne yapıyor? Method'a her zaman **değerin kopyası** gider. Primitive'de kopya değer → orijinal değişmez. Reference'da kopyalanan şey **adres** → method aynı array'i/object'i değiştirebilir.

Constructor

Tanım	Kurallar	Türler
new ile object oluşurken **otomatik** çalışan özel blok	Adı **class adıyla birebir aynı**	**Default**: compiler ekler
Görevi: field'lara ilk değer vermek	**Return type yok** (void bile yazılmaz)	**No-arg**: programcının yazdığı parametresiz
Hiç yazılmazsa compiler default ekler	**abstract, static, final, synchronized olamaz**	**Parameterized**: parametre alır
—	Access modifier alabilir (public, protected, default, private)	**Copy**: aynı class'tan object alıp kopyalar

Return type yazılırsa constructor değildir:

```java
class Test {
    void Test() { System.out.println("Ben bir method'um"); }   // method!
}
Test t = new Test();    // çıktı yok
t.Test();               // Ben bir method'um (açıkça çağrılırsa)
```

Ne yapıyor? void Test() **derlenir ama method'dur**; new ile çalışmaz. Return type olmadan Test() { ... } yazılırsa constructor olur ve new ile otomatik çağrılır.

Default ve no-arg constructor

```java
class Bike {
    Bike() { System.out.println("Bike is created"); }   // no-arg
}
class Student { int id; String name; }   // constructor yok → compiler default ekler
Bike b = new Bike();                     // Bike is created
Student s = new Student();
System.out.println(s.id + " " + s.name); // 0 null
```

Ne yapıyor? Default constructor: class'ta hiç constructor yoksa compiler'ın eklediği boş, parametresiz constructor. 0 ve null constructor'dan değil, **field'ların default initialization'ından** gelir. new her zaman argümanlarla eşleşen constructor'ı çağırır.

Parameterized constructor ve overloading

```java
class Student {
    int id, age;
    String name;
    Student(int id, String name) { this.id = id; this.name = name; }
    Student(int id, String name, int age) {
        this(id, name);               // constructor chaining
        this.age = age;
    }
    void display() { System.out.println(id + " " + name + " " + age); }
}
new Student(111, "Karan").display();      // 111 Karan 0
new Student(222, "Aryan", 25).display();  // 222 Aryan 25
```

Ne yapıyor? Parameterized constructor her object'e farklı başlangıç verir. **Overloading**: parametre sayısı/tipi farklı birden fazla constructor. **this(...)** aynı class'taki başka constructor'ı çağırır (**constructor chaining**). İlk object'te age verilmediği için 0.

Sık hata: default constructor kaybolur

```
class Point {
    int x, y;
    Point(int x, int y) { this.x = x; this.y = y; }
}
Point p = new Point();   // HATA: constructor Point cannot be applied to given types
```

Ne yapıyor? Herhangi bir constructor yazılınca compiler default eklemez. Çözüm: no-arg constructor'ı da yaz → Point() { this(0, 0); } → new Point() çalışır, çıktı 0,0.

Copy constructor

```
class Student {
    int id;
    String name;
    Student(int id, String name) { this.id = id; this.name = name; }
    Student(Student other) {          // copy constructor
        this.id = other.id;
        this.name = other.name;
    }
    void display() { System.out.println(id + " " + name); }
}
Student s1 = new Student(111, "Karan");
Student s2 = new Student(s1);         // yeni, bağımsız object
s2.name = "Aryan";
s1.display();                         // 111 Karan
s2.display();                         // 111 Aryan
```

Ne yapıyor? Java'da C++'taki gibi otomatik copy constructor **yok**; programcı yazar. s2 = s1 kopya değildir (aynı object); copy constructor **yeni object** üretir, bu yüzden s2 değişince s1 etkilenmez. Alternatif: clone() veya field'ları tek tek atamak.

Constructor vs method

Constructor	Method
Object'in durumunu başlatır	Object'in davranışını tanımlar
Return type **yok** (void bile değil)	Mutlaka return type var (void dahil)
new ile **otomatik** çağrılır	Açıkça çağrılır: obj.method()
Hiç yoksa compiler default ekler	Compiler method eklemez
Adı class adıyla aynı olmak zorunda	Herhangi geçerli isim
Inherit edilmez	Inherit edilir, override edilebilir

PDF 4 alıştırma cevapları

S1. `Box a = new Box(); Box b = a; b.v = 7; System.out.println(a.v);`

- Cevap: 7
- Neden: b = a yeni object oluşturmaz; ikisi aynı object'i gösterir (aliasing).

S2. `Box a = new Box(); Box b = new Box(); System.out.println(a == b);`

- Cevap: false
- Neden: İki ayrı new → iki farklı object; == adres karşılaştırır.

S3. `class T { T() { print("C "); } void T() { print("M "); } } new T();`

- Cevap: C
- Neden: T() constructor, new ile çalışır. void T() return type'ı olduğu için method; çağrılmadığı için M yazılmaz.

S4. `static void inc(int x) { x++; } int n = 1; Num.inc(n); System.out.println(n);`

- Cevap: 1
- Neden: Pass-by-value: method n'in kopyasını artırır, orijinal değişmez.

PDF 5 · static ve this

static: class'a ait, herkesin ortak malı

- static üye **object'e değil class'a** aittir; bellekte **tek kopya**.

- Kullanıldığı yerler: variable (class variable), method (class method), block, nested class.
- Object oluşturmadan erişilir: **ClassName.member**.
- static variable class yüklenirken **bir kez** oluşur ve default değer alır.
- **main static'tir**: JVM programı object oluşturmadan başlatabilsin diye.

static variable

```
class Student {
    int rollno;                     // instance
    String name;
    static String college = "ITS";   // static
    Student(int r, String n) { rollno = r; name = n; }
    void display() { System.out.println(rollno + " " + name + " " + college); }
}
Student s1 = new Student(111, "Karan");
Student s2 = new Student(222, "Aryan");
s1.display();                   // 111 Karan ITS
Student.college = "BBDIT";      // herkes için değişir
s1.display();                   // 111 Karan BBDIT
s2.display();                   // 222 Aryan BBDIT
```

Ne yapıyor? Her object kendi rollno/name'ine sahip, college'ı **paylaşır**. Ortak özellikler için kullanılır (okul, şirket adı, faiz oranı); her object'te ayrı kopya olmadığı için **bellek tasarrufu**. Tek satır değişiklik tüm object'leri etkiler.

Sayaç: instance mı, static mi?

```
class Counter {
    int count = 0;            // instance → 1 1 1
    // static int count = 0;  // static   → 1 2 3
    Counter() { count++; System.out.println(count); }
}
new Counter(); new Counter(); new Counter();
```

Ne yapıyor? Instance'ta her object kendi count'unu 0'dan başlatır → 1, 1, 1. static'te tüm object'ler **aynı** count'u artırır → 1, 2, 3.

static method

```
class Student {
    int rollno; String name;
    static String college = "ITS";
    static void change() { college = "BBDIT"; }   // static method
    Student(int r, String n) { rollno = r; name = n; }
    void display() { System.out.println(rollno + " " + name + " " + college); }
}
class Calculate {
    static int cube(int x) { return x * x * x; }   // utility
}
Student.change();                          // object gerekmez
new Student(111, "Karan").display();       // 111 Karan BBDIT
System.out.println(Calculate.cube(5));     // 125
```

Ne yapıyor? static method object olmadan, **class adıyla** çağrılır; static variable'ları okuyup değiştirebilir. Tipik kullanım: **utility** method'lar (Math.max, Integer.parseInt).

Kısıtlama:

```
class A {
    int a = 40;                                  // non-static
    public static void main(String[] args) {
        // System.out.println(a);  -> HATA: non-static variable a cannot be referenced from a static context
        A obj = new A();
        System.out.println(obj.a);               // 40
    }
}
```

Ne yapıyor? static method **hangi object'in** a'sını kullanacağını bilemez; instance üyeye bir **object üzerinden** erişilir. **this ve super de static context'te kullanılamaz.**

static block ve çalışma sırası

```java
public class Main {
    static int max;
    static { System.out.println("1) static block"); max = 100; }
    { System.out.println("3) instance block"); }
    Main() { System.out.println("4) constructor"); }
    public static void main(String[] args) {
        System.out.println("2) main, max = " + max);
        new Main();
    }
}
// 1) static block · 2) main, max = 100 · 3) instance block · 4) constructor
```

Block	Ne zaman çalışır
static block	Class belleğe yüklenirken **bir kez**, **main'den önce**; static variable'ları hazırlar
Instance block	Her object oluşturulduğunda, **constructor'dan önce**
Constructor	Instance block'tan sonra

JDK 7'den beri main olmadan sadece static block ile program **çalıştırılamaz**.

static nested class

```java
class Library {
    static int bookCount = 0;
    static class Book {                     // static nested class
        String title;
        Book(String title) { this.title = title; bookCount++; }
        void info() { System.out.println("Kitap: " + title); }
    }
}
Library.Book b1 = new Library.Book("Java 101");
Library.Book b2 = new Library.Book("OOP");
b1.info();                                  // Kitap: Java 101
System.out.println("Toplam: " + Library.bookCount);   // Toplam: 2
```

Ne yapıyor? Outer class'ın object'i **olmadan** oluşturulur: new Library.Book(...). Outer'ın yalnızca **static** üyelerine doğrudan erişebilir. İlgili class'ları mantıksal gruplamak için.

Özellik	static	Instance
Kime ait?	Class'a	Her object'e
Kopya sayısı	Tek	Object başına bir
Oluşma	Class yüklenince	new ile
Erişim	ClassName.member	obj.member
Diğer tür üyeye erişim	Instance üyeye **doğrudan erişemez**	static üyelere erişebilir
this / super	**Kullanılamaz**	Kullanılabilir
Overriding	**Override edilemez** (method hiding)	Override edilebilir
Tipik kullanım	Sabit, sayaç, utility method	Object'e özgü durum ve davranış

this: 6 kullanım

#	Kullanım	Örnek
1	Instance variable: parametreyle aynı isimdeki field'ı ayırt eder	this.name = name;
2	Method çağırma (yazılmasa da compiler ekler)	this.m()
3	this(): aynı class'taki başka constructor (chaining)	this(id, name);
4	Method argümanı: mevcut object'i gönderir	printer.print(this)
5	Constructor argümanı	new B(this)
6	return this: method chaining	obj.a().b().c()

this ile field ayırt etme

```java
class Student {
    int rollno; String name;
    Student(int rollno, String name) {
        rollno = rollno;          // ✗ parametre kendine atanır
        name = name;
        // ✓ this.rollno = rollno; this.name = name;
    }
}
Student s = new Student(111, "Ankit");
System.out.println(s.rollno + " " + s.name);   // ✗ 0 null  /   ✓ 111 Ankit
```

Ne yapıyor? this yoksa local parametre field'ı **gölgeler (shadowing)**; parametre kendine atanır, field'lar default değerde kalır (0 null). this.rollno → field, rollno → parametre.

this() ile constructor chaining

```java
class Student {
    int rollno; String name; float fee;
    Student(int rollno, String name) { this.rollno = rollno; this.name = name; }
    Student(int rollno, String name, float fee) {
        this(rollno, name);           // ortak kodu tekrar kullan
        this.fee = fee;
    }
    void display() { System.out.println(rollno + " " + name + " " + fee); }
}
new Student(111, "ankit").display();          // 111 ankit 0.0
new Student(112, "sumit", 6000f).display();   // 112 sumit 6000.0
```

Ne yapıyor? Ortak başlatma kodu tek constructor'da; tekrar önlenir. **this(...) constructor'ın ilk satırı olmalı**; ikinci satıra yazılırsa: "call to this must be first statement in constructor" hatası. (Box(int w, int h) { this(w); this.h = h; } → 3x4.)

this'i argüman olarak göndermek

```java
class Printer {
    void print(Book b) { System.out.println("Yazdırılıyor: " + b.title); }
}
class Book {
    String title;
    Book(String title) { this.title = title; }
    void printWith(Printer p) { p.print(this); }   // kendini gönder
}
Book book = new Book("Java OOP");
book.printWith(new Printer());    // Yazdırılıyor: Java OOP
```

Ne yapıyor? Object kendisini başka object'in method'una verir. Constructor'a da verilebilir: new Helper(this). Event handling'de (butona dinleyici olarak kendini vermek) çok kullanılır.

return this: method chaining

```java
class Counter {
    int value;
    Counter add(int n) { value += n; return this; }   // kendini döndür
    Counter reset() { value = 0; return this; }
}
Counter c = new Counter().add(5).add(10).add(2);
System.out.println(c.value);                  // 17
System.out.println(c.reset().add(3).value);   // 3
StringBuilder sb = new StringBuilder();
sb.append("Nesne ").append("Tabanlı ").append("Programlama");
System.out.println(sb);                       // Nesne Tabanlı Programlama
```

Ne yapıyor? Return type **class'ın kendisi** olur ve return this; yazılır; çağrılar art arda zincirlenir (**fluent API**). JDK örneği: StringBuilder.append().

this gerçekten mevcut object mi?

```java
public class Main {
    void check(Main other) { System.out.println(this == other); }
    void hello() { System.out.println("hello"); }
    void run() { hello(); this.hello(); }   // compiler hello()'yu this.hello() yapar
    public static void main(String[] args) {
        Main obj = new Main();
        obj.check(obj);          // true  aynı object
        obj.check(new Main());   // false farklı object
        obj.run();               // hello · hello
    }
}
```

Ne yapıyor? Method'u çağıran object obj ise method içindeki this de **o object**. println(obj) ile method içinde println(this) aynı değeri (Main@1b6d3586 benzeri) yazar. this static method'da kullanılamaz.

PDF 5 · Overloading, Overriding, super

Method overloading

- Aynı class'ta **aynı isimde** ama **farklı parametre listesine** sahip birden fazla method.
- Parametreler **sayı, tip veya sıra** ile farklılaşır.
- **Sadece return type farkı yetmez** → compile error.
- Karar **compile time'da** verilir → **compile-time (static) polymorphism**.
- a(int,int) ve b(int,int,int) yerine tek isim add(...) → okunabilirlik.

```java
class Adder {
    static int add(int a, int b) { return a + b; }
    static int add(int a, int b, int c) { return a + b + c; }
    static double add(double a, double b) { return a + b; }
    static String add(String a, String b) { return a + b; }
}
class Printer {
    static void show(String s, int n) { System.out.println("S,I"); }
    static void show(int n, String s) { System.out.println("I,S"); }
}
System.out.println(Adder.add(11, 11));          // 22
System.out.println(Adder.add(11, 11, 11));      // 33
System.out.println(Adder.add(12.5, 12.25));     // 24.75
System.out.println(Adder.add("Java", "!"));     // Java!
Printer.show("a", 1);                           // S,I
Printer.show(1, "a");                           // I,S
```

Farklılık	Örnek
Parametre **sayısı**	add(int,int) / add(int,int,int)
Parametre **tipi**	add(int,int) / add(double,double)
Parametre **sırası**	show(String,int) / show(int,String)

Sadece return type farkı:

```java
static int add(int a, int b) { return a + b; }
static double add(int a, int b) { return a + b; }   // HATA: method add(int,int) is already defined
```

Ne yapıyor? add(11, 11) çağrısında hangisi seçilecek? Compiler bilemez. JDK örneği: System.out.println(5), ('A'), (2.5), (true), ("Java"), () → **PrintStream'de 10 farklı println** overload'u var.

Type promotion

```java
class Calc {
    void sum(int a, long b) { System.out.println("int,long"); }
    void sum(long a, long b) { System.out.println("long,long"); }
    void show(int a) { System.out.println("int"); }
    void show(double a) { System.out.println("double"); }
}
Calc c = new Calc();
c.sum(20, 20);    // int,long  (tam eşleşme yok → promotion)
c.show(5);        // int       (tam eşleşme)
c.show(5L);       // double    (long → double)
c.show('A');      // int       (char → int)
c.show(2.5f);     // double    (float → double)
```

Ne yapıyor? Tam eşleşme yoksa argüman genişletilir: **byte → short → int → long → float → double, char → int**. Birden

fazla aday varsa **en spesifik** olan seçilir (sum(20,20)'de int,long; long,long'dan daha yakın). **Daraltma (double → int) asla otomatik yapılmaz.**

Ambiguity:

```
void sum(int a, long b) { System.out.println("a"); }
void sum(long a, int b) { System.out.println("b"); }
new Calc().sum(20, 20);    // HATA: reference to sum is ambiguous
new Calc().sum(20, 20L);   // a  (ikinci argüman zaten long → tam eşleşme)
```

Ne yapıyor? İki method eşit derecede uygun → compiler seçemez. Çözüm: argümanı açıkça belirt (20L).

main() de overload edilebilir

```
public class Main {
    public static void main(String[] args) {
        System.out.println("main with String[]");
        main("test");
        main();
    }
    public static void main(String arg) { System.out.println("main with String"); }
    public static void main() { System.out.println("main without args"); }
}
```

Ne yapıyor? JVM yalnızca **public static void main(String[] args)** imzasını giriş noktası kullanır. Diğerleri sıradan overload; ancak **açıkça çağrılırsa** çalışır.

Avantaj	Dikkat
Okunabilirlik: benzer işler aynı isimle	Type promotion ambiguity hatası
Esnek API (println, Math.max)	Çok fazla overload anlaşılmayı zorlaştırır
Compile-time polymorphism	Overload'lar aynı anlamda iş yapmalı
Performans maliyeti yok (seçim derlemede)	Hangi versiyonun çağrıldığı açık olmayabilir: dokümante et

Method overriding

```
class Vehicle { void run() { System.out.println("Vehicle is running"); } }
class Bike extends Vehicle {
    @Override
    void run() { System.out.println("Bike is running"); }
}
new Bike().run();     // Bike is running  (override olmasaydı: Vehicle is running)
```

Ne yapıyor? Aynı isim + aynı parametre + IS-A ilişkisi = overriding. Subclass parent'ın method'unu yeniden tanımlar; override yoksa parent'ınki miras alınıp çalışır.

Kural	Açıklama
İmza aynı	Aynı ad, aynı parametre sayısı ve tipleri; IS-A gerekli
Return type	Aynı tip veya alt tipi (**covariant return type**, Java 5+); primitive'de birebir aynı
Access modifier	**Daha kısıtlayıcı olamaz**: public → yalnızca public; protected → protected/public
Override edilemeyenler	**final ve private** method'lar, **static** method'lar (hiding olur), **constructor**'lar

Banka faiz örneği

```
class Bank { int getRateOfInterest() { return 0; } }
class SBI extends Bank { @Override int getRateOfInterest() { return 8; } }
class ICICI extends Bank { @Override int getRateOfInterest() { return 7; } }
class AXIS extends Bank { @Override int getRateOfInterest() { return 9; } }
Bank[] banks = { new SBI(), new ICICI(), new AXIS() };
for (Bank b : banks) {
    System.out.print(b.getClass().getSimpleName());
    System.out.println(": %" + b.getRateOfInterest());
}
// SBI: %8 · ICICI: %7 · AXIS: %9
```

Ne yapıyor? Reference tipi Bank, object tipi SBI/ICICI/AXIS. Çalışacak method **runtime'da object'in gerçek tipine** göre seçilir: **dynamic method dispatch**. getClass().getSimpleName() class adını verir. Yeni banka için yalnızca yeni subclass yazmak yeter.

@Override neden önemli?

```
class Vehicle { void run() { System.out.println("Vehicle"); } }
class Bike extends Vehicle {
    void Run() { System.out.println("Bike"); }   // büyük R → YENİ method!
}
Vehicle v = new Bike();
v.run();   // Vehicle
```

Ne yapıyor? Yazım hatası yüzünden override olmamış; **derlenir ama logic error**. @Override eklenirse compiler yakalar: "method does not override or implement a method from a supertype". Her override'da @Override yaz.

Access kısıtlanamaz: A'da protected void msg() → B'de void msg() (default, daha kısıtlı) → **HATA: weaker access privileges**. B'de public void msg() → geçerli, çıktı "Hello B".

static method'lar override edilmez: method hiding

```
class Parent {
    static void hello() { System.out.println("Parent static"); }
    void hi() { System.out.println("Parent instance"); }
}
class Child extends Parent {
    static void hello() { System.out.println("Child static"); }
    @Override void hi() { System.out.println("Child instance"); }
}
Parent p = new Child();
p.hello();   // Parent static   (reference tipine bakılır, compile time)
p.hi();      // Child instance  (object tipine bakılır, runtime)
```

Ne yapıyor? Aynı imzalı static method parent'ınkini **gizler (method hiding)**; static çağrı **reference tipine** göre compile time'da çözülür, polymorphism yok. Bu yüzden main() de override edilemez.

Kriter	Overloading	Overriding
Nerede?	Aynı class (veya miras alınanla)	Parent–child arasında (IS-A)
Parametreler	**Farklı**	**Aynı**
Return type	Serbest (tek başına yetmez)	Aynı veya covariant
Access modifier	Serbest	Daha kısıtlayıcı olamaz
Polymorphism	**Compile-time** (static binding)	**Runtime** (dynamic binding)
static method	Overload edilebilir	Override edilemez (hiding)
private / final	Overload edilebilir	Override edilemez
Exception	Serbest	Daha geniş checked exception fırlatamaz

super: 3 kullanım

Kullanım	Ne yapar
super.field	Parent'ın field'ına erişir; child aynı isimde field ile gizlediyse
super.method()	Parent'ın method'unu çağırır; override edilmiş method'un parent versiyonu gerektiğinde, davranışı **genişletmek** için
super(...)	Parent'ın constructor'ını çağırır; **ilk satır** olmalı; yazılmazsa compiler **super()** ekler

```
class Animal {
    String color = "white";
    void eat() { System.out.println("eating..."); }
}
class Dog extends Animal {
    String color = "black";                  // parent'ınkini gizler
    @Override void eat() { System.out.println("eating bread..."); }
    void show() {
        System.out.println(color);           // black   (Dog'un)
        System.out.println(super.color);     // white   (Animal'ın)
        eat();                               // eating bread...
        super.eat();                         // eating...
    }
}
new Dog().show();
```

Ne yapıyor? Öncelik her zaman **en yakındaki (child)** üyede; super. önekiyle parent versiyonuna ulaşılır. Parent ile aynı isimde field (**field hiding**) kafa karıştırır, önerilmez.

super() ve constructor sırası

```
class Animal { Animal() { System.out.println("Animal()"); } }
class Dog extends Animal {
    Dog() {
        // super();  yazılsa da yazılmasa da çağrılır
        System.out.println("Dog()");
    }
}
new Dog();   // Animal() · Dog()

class A { A() { System.out.println("A constructor"); } }
class B extends A { B() { System.out.println("B constructor"); } }
class C extends B { C() { System.out.println("C constructor"); } }
new C();     // A constructor · B constructor · C constructor
```

Ne yapıyor? new C() → C'nin constructor'ı önce super() ile B'yi, B de A'yı çağırır. **Parent kısım önce, child sonra** hazırlanır (yukarıdan aşağıya). Bellekte **tek bir C object'i** oluşur; içinde A ve B'den gelen field'lar da var.

super(...) ve super.method() birlikte

```
class Person {
    int id; String name;
    Person(int id, String name) { this.id = id; this.name = name; }
    void display() { System.out.print(id + " " + name); }
}
class Emp extends Person {
    float salary;
    Emp(int id, String name, float salary) {
        super(id, name);                        // parent'ın constructor'ı
        this.salary = salary;
    }
    @Override void display() {
        super.display();                        // parent'ın işini yap
        System.out.println(" " + salary);       // üstüne ekle
    }
}
new Emp(1, "ankit", 45000f).display();          // 1 ankit 45000.0
```

Ne yapıyor? super(id, name) parent'ın başlatma kodunu **tekrar kullanır**; super.display() ile davranış baştan yazılmaz, **genişletilir**.

Sık hata: Person'da yalnızca Person(String name) varsa Emp() { } içindeki **örtük super()** eşleşmez → "constructor Person cannot be applied to given types". Çözüm: Emp(String name) { super(name); } → new Emp("Ali").name → Ali.

this vs super

Kullanım	this	super
Neyi gösterir?	Mevcut object (kendi class'ı)	Mevcut object'in parent'tan gelen kısmı
Field	this.name	super.name
Method	this.display()	super.display()
Constructor	this(...) → aynı class	super(...) → parent
Konum	Constructor'ın ilk satırı	Constructor'ın ilk satırı
Birlikte	**this() ve super() aynı constructor'da birlikte olamaz**	
static context	Kullanılamaz	Kullanılamaz

PDF 5 alıştırma cevapları

S1. `class K { static int n; K() { n++; } } new K(); new K(); new K(); System.out.println(K.n);`

- Cevap: 3
- Neden: n static, tüm object'ler aynı n'i artırır.

S2. `void f(int x) { print("I"); } void f(long x) { print("L"); } new P().f(5); new P().f(5L);`

- Cevap: IL
- Neden: 5 int literal → f(int) tam eşleşme. 5L long → f(long).

S3. `class A { int v = 1; } class B extends A { int v = 2; } A x = new B(); System.out.println(x.v);`

- Cevap: 1
- Neden: **Field'lar override edilmez** (field hiding); erişim **reference tipine** (A) göre çözülür. Polymorphism yalnızca method'larda.

S4. `class A { A() { print("1"); } } class B extends A { B() { print("2"); } } class C extends B { } new C();`

- Cevap: 12
- Neden: C'nin compiler'ın eklediği default constructor'ı super() → B() → super() → A(). Önce A "1", sonra B "2"; C bir şey yazmaz.

PDF 6 · Inheritance ve Aggregation

Inheritance nedir?

- Bir class'ın başka bir class'ın **field ve method'larını devralması**.
- Devralan: **subclass (child)**; devredilen: **superclass (parent)**.
- **IS-A** ilişkisini modeller: "Programmer bir Employee'dir".
- Java'da **extends** ile yapılır.
- Subclass yeni field/method ekleyebilir ve miras aldığı method'ları **override** edebilir.
- **Neden?** Code reusability, method overriding (runtime polymorphism), genişletilebilir tasarım.
- UML: **içi boş üçgen ok** parent'ı gösterir.

Terim	Diğer adları	Açıklama
Superclass	Parent, base class	Özellikleri devredilen
Subclass	Child, derived, extended class	Başka class'tan türeyen
extends	—	class B extends A
IS-A	Parent–child ilişkisi	"B bir A'dır" cümlesi anlamlıysa inheritance uygun
Reusability	Kodun yeniden kullanımı	Parent'taki kod subclass'ta tekrar yazılmaz

Programmer IS-A Employee

```
class Employee {
    float salary = 40000;
    void work() { System.out.println("Çalışıyor..."); }
}
class Programmer extends Employee {
    int bonus = 10000;                                  // yeni field
    void writeCode() { System.out.println("Kod yazıyor..."); }  // yeni method
}
Programmer p = new Programmer();
System.out.println("Programmer salary is: " + p.salary);  // 40000.0
System.out.println("Bonus of Programmer is: " + p.bonus); // 10000
p.work();        // Çalışıyor...    (Employee'den miras)
p.writeCode();   // Kod yazıyor...  (kendi method'u)
```

Ne yapıyor? Programmer object'i hem kendi hem Employee'nin üyelerine erişir. salary **float** olduğu için **40000.0** yazılır. Employee'de hiçbir değişiklik yapılmadı → reusability.

Neler miras alınır?

Miras alınır	Miras alınmaz / doğrudan erişilemez
public ve protected field/method	**private** üyeler: object'te **vardır** ama subclass doğrudan erişemez
Aynı package'taysa **default** üyeler	**Constructor**'lar miras alınmaz; **super(...)** ile çağrılır
static üyeler subclass üzerinden erişilebilir (override edilmez)	Instance initializer block'lar

```java
class Account {
    private double balance = 100;      // sadece Account görür
    protected String owner = "Ali";    // subclass görür
    public double getBalance() { return balance; }
}
class Savings extends Account {
    void show() {
        System.out.println(owner);         // Ali   OK: protected
        // System.out.println(balance);    // HATA: private
        System.out.println(getBalance());  // 100.0 public getter ile
    }
}
new Savings().show();
```

Ne yapıyor? balance Savings object'inde **var** ama Savings kodu ona dokunamaz; public getter ile okunur. Encapsulation korunur. Subclass'ın erişmesi gereken üyeler için **protected**.

java.lang.Object: her class'ın atası

```java
class Student {                        // gizlice: extends Object
    String name;
    Student(String name) { this.name = name; }
    @Override
    public String toString() { return "Student[" + name + "]"; }
}
Student s = new Student("Ayşe");
System.out.println(s);                 // Student[Ayşe]  (toString() çağrılır)
System.out.println(s instanceof Object);                    // true
System.out.println(s.getClass().getSuperclass().getName());    // java.lang.Object
```

Ne yapıyor? extends yazılmayan **her class otomatik Object'ten** türer; bu yüzden her object'te **toString(), equals(), hashCode(), getClass()** var. println(s) otomatik toString() çağırır. Override edilmezse **Student@1b6d3586** gibi çıktı. getSuperclass().getName() parent'ın tam adını verir.

Inheritance türleri

Tür	Yapı	Class ile?
Single	B, A'dan türer	✓
Multilevel	C → B → A zinciri	✓
Hierarchical	B ve C aynı A'dan	✓
Multiple	C hem A hem B'den	✗ class ile · ✓ **interface ile**
Hybrid	Birden fazla türün birleşimi	Ancak multiple içermezse

```java
// Single
class Animal { void eat() { System.out.println("eating..."); } }
class Dog extends Animal { void bark() { System.out.println("barking..."); } }
Dog d = new Dog();
d.bark();   // barking...
d.eat();    // eating...
```

Ne yapıyor? Bir subclass **yalnızca bir** superclass'tan türer; en yaygın ve en basit biçim.

```java
// Multilevel
class BabyDog extends Dog { void weep() { System.out.println("weeping..."); } }
BabyDog b = new BabyDog();
b.weep();   // weeping...  BabyDog
b.bark();   // barking...  Dog'dan
b.eat();    // eating...   Animal'dan (dolaylı)
System.out.println(b instanceof Animal);   // true
```

Ne yapıyor? BabyDog → Dog → Animal zinciri; en alttaki class zincirdeki tüm üyelere sahip. Çok derin zincirler (4-5+ seviye) anlaşılmayı zorlaştırır; sığ tut.

```java
// Hierarchical
class Cat extends Animal { void meow() { System.out.println("meowing..."); } }
Cat c = new Cat();
c.meow();   // meowing...
c.eat();    // eating...
// c.bark(); -> HATA: Cat, Dog'un method'unu bilmez
Animal[] pets = { new Dog(), new Cat() };
for (Animal a : pets) a.eat();   // eating... eating...
```

Ne yapıyor? Birden fazla subclass aynı superclass'tan türer. **Kardeş class'lar** (Dog, Cat) birbirinin üyelerini **görmez**. Ortak parent sayesinde hepsi Animal array'inde tutulabilir.

Diamond problem

D hem B'den hem C'den türeseydi ve ikisinde de msg() olsaydı (B → Hello, C → Welcome), new D().msg() hangisini çalıştıracak **belirsiz** olurdu. Şekil elmasa benzediği için **diamond problem**. Java dili basit tutmak için **class'larda multiple inheritance'a izin vermez**: class C extends A, B → compile error ("'{' expected"). **Çözüm: bir class birden fazla interface implement edebilir.**

```
interface Fighter { void attack(); }
interface Weapon { void use(); }
class Warrior implements Fighter, Weapon {
    public void attack() { System.out.println("Kılıçla saldırır"); }
    public void use() { System.out.println("Kılıcı kullanır"); }
}
Warrior w = new Warrior();
w.attack();   // Kılıçla saldırır
w.use();      // Kılıcı kullanır
```

Ne yapıyor? implements ile iki interface; Warrior her iki interface'in method'larını **public** olarak doldurur (interface method'ları public olduğu için).

Interface'lerde diamond çözümü (Java 8+):

```
interface A { default void hello() { System.out.println("A"); } }
interface B { default void hello() { System.out.println("B"); } }
class C implements A, B {
    @Override
    public void hello() {        // ZORUNLU: çakışmayı çöz
        A.super.hello();         // A'nınkini çağır
        B.super.hello();         // B'ninkini çağır
        System.out.println("C");
    }
}
new C().hello();   // A · B · C
```

Ne yapıyor? Java 8 ile interface'ler **default method** (gövdeli) içerebilir. İki interface aynı default method'u getirirse compiler class'ı **override etmeye zorlar**. **InterfaceAdı.super.method()** ile istenen versiyon seçilir.

Hybrid

```
class C { public void disp() { System.out.println("C"); } }
class A extends C { public void disp() { System.out.println("A"); } }
class B extends C { public void disp() { System.out.println("B"); } }
class D extends A { public void disp() { System.out.println("D"); } }
C[] objs = { new C(), new A(), new B(), new D() };
for (C o : objs) o.disp();                  // C · A · B · D
System.out.println(new D() instanceof C);   // true
```

Ne yapıyor? Hierarchical (A ve B, C'den) + multilevel (D → A → C) bir arada. Multiple içermediği için class'larla yazılabilir; her object **kendi** disp()'ini çalıştırır. Multiple içeren hybrid interface'lerle kurulur.

Shape hiyerarşisi

```
class Shape {
    String name;
    Shape(String name) { this.name = name; }
    double area() { return 0; }
    void describe() { System.out.printf("%s alanı: %.2f%n", name, area()); }
}
class Circle extends Shape {
    double r;
    Circle(double r) { super("Daire"); this.r = r; }
    @Override double area() { return Math.PI * r * r; }
}
class Rectangle extends Shape {
    double w, h;
    Rectangle(double w, double h) { super("Dikdörtgen"); this.w = w; this.h = h; }
    @Override double area() { return w * h; }
}
// main: Shape[] shapes = { new Circle(1), new Rectangle(3, 4) }; describe() + toplam
// Daire alanı: 3.14 · Dikdörtgen alanı: 12.00 · Toplam: 15.14
```

Kod	Ne yapıyor?
super("Daire")	Parent constructor'ına isim gönderir
@Override area()	Her subclass yalnızca **farklı olanı** (alan formülü) yazar
describe()	Ortak kod bir kez parent'ta; area() runtime'da doğru subclass'tan çağrılır
printf("%s alanı: %.2f%n")	%s metin, %.2f iki ondalık, %n satır sonu

Upcasting, downcasting, instanceof

```
class Animal { void eat() { System.out.println("eating"); } }
class Dog extends Animal { void bark() { System.out.println("woof"); } }
class Cat extends Animal { }
Animal a = new Dog();           // upcasting: otomatik
a.eat();                        // eating
// a.bark();                    // HATA: reference tipi Animal
if (a instanceof Dog) {
    Dog d = (Dog) a;            // downcasting: açık cast
    d.bark();                   // woof
}
Animal b = new Cat();
if (b instanceof Dog dog) dog.bark();     // pattern matching (Java 16+)
else System.out.println("b bir Dog değil");
// Dog x = (Dog) b;             // ClassCastException!
```

Kavram	Ne yapar
Upcasting	Subclass → superclass reference; **her zaman güvenli, otomatik**
Reference tipi	**Hangi method'ların çağrılabileceğini** belirler (a.bark() hata)
Downcasting	Superclass → subclass; **açık cast** gerekir
instanceof	Object o tipte mi? Downcast'ten önce kontrol
instanceof Dog dog	Pattern matching (Java 16+): kontrol + cast tek adımda
Yanlış tipe cast	**ClassCastException** (runtime)

final ile sınırlamak

```
final class Utility { static int twice(int x) { return 2 * x; } }
class MyUtil extends Utility { }      // HATA: cannot inherit from final Utility

class Account { final void audit() { } }
class Fake extends Account {
    @Override void audit() { }        // HATA: overridden method is final
}
```

Ne yapıyor? final class extend edilemez (JDK örnekleri: **String, Integer, Math**); davranışı değiştirilmesin istenen class'lar için. **final method override edilemez**; güvenlik açısından kritik method'lar (doğrulama, denetim) kilitlenir.

Inheritance ne zaman?

Kullan	Kaçın
"B **bir** A'dır" her zaman doğruysa (IS-A)	Sadece birkaç method'u tekrar kullanmak için extends
Subclass parent'ın **yerine** kullanılabiliyorsa	"Car extends Engine" gibi IS-A olmayan ilişkiler
Ortak davranış gerçekten paylaşılıyorsa	Çok derin hiyerarşiler → bu durumlarda **composition / aggregation (HAS-A)**

Aggregation (HAS-A)

- Bir class'ın başka class'ın object'ine **reference (entity reference) tutması**.
- **HAS-A**: "Employee'nin bir Address'i vardır".
- İçerilen object **bağımsız yaşayabilir**, başka object'lerle **paylaşılabilir**.
- Inheritance olmadan kodu yeniden kullanır: iş, içerilen object'e **delege edilir**.
- Association'ın **zayıf bağlı** özel hâli.
- UML: **içi boş elmas (◇)** aggregation, **içi dolu elmas (◆)** composition.

```
class Operation { int square(int n) { return n * n; } }
class Circle {
    Operation op;                                  // HAS-A
    Circle(Operation op) { this.op = op; }
    double area(int radius) { return Math.PI * op.square(radius); }   // delegation
}
Operation op = new Operation();
Circle c1 = new Circle(op);      // aynı Operation
Circle c2 = new Circle(op);      // paylaşılıyor
System.out.println(c1.area(5));  // 78.53981633974483
System.out.println(c2.area(1));  // 3.141592653589793
```

Ne yapıyor? Circle kare alma işini kendisi yazmak yerine Operation'a **delege eder**. Operation dışarıda oluşturulup verildiği için iki Circle **paylaşır**. π için 3.14 yerine **Math.PI** daha doğru.

Employee HAS-A Address

```
class Address {
    String city, country;
    Address(String city, String country) { this.city = city; this.country = country; }
}
class Emp {
    int id; String name;
    Address addr;                              // Emp HAS-A Address
    Emp(int id, String name, Address addr) { this.id = id; this.name = name; this.addr = addr; }
    void display() {
        System.out.println(id + " " + name);
        System.out.println(" " + addr.city + ", " + addr.country);
    }
}
// main: Address gms = new Address("Gümüşhane", "Türkiye");
//       e1 = new Emp(111, "Varun", gms); e2 = new Emp(112, "Arun", gms);
//       e1.display(); e2.display(); gms.city = "Torul"; e2.display();
// 111 Varun Gümüşhane, Türkiye · 112 Arun Gümüşhane, Türkiye · 112 Arun Torul, Türkiye
```

Ne yapıyor? İki çalışan **aynı Address object'ini** alıyor; gms.city = "Torul" yapınca ikisi de yeni adresi görür. Çalışan silinse de adres yaşamaya devam eder → **aggregation**.

IS-A mı, HAS-A mı?

```
class Engine { void start() { System.out.println("Motor çalıştı"); } }
// ✗ class Car extends Engine { }   → derlenir, çalışır ama modelleme yanlış
// ✓
class Car {
    private Engine engine = new Engine();
    void start() { engine.start(); }
}
new Car().start();   // Motor çalıştı
```

Ne yapıyor? Araba bir motor **değildir**. extends'te Car, motorun tüm public method'larını dışarı açar. HAS-A'da Car yalnızca istediği davranışı sunar ve motor kolayca değiştirilebilir (ör. ElectricEngine).

Kriter	Inheritance	Aggregation	Composition
İlişki	IS-A	HAS-A (zayıf)	HAS-A (güçlü)
Java'da	extends / implements	**Dışarıdan verilen** reference	**İçeride oluşturulan** reference
Coupling	En yüksek	Düşük	Orta
Parçanın ömrü	—	Bütünden **bağımsız**	Bütüne **bağlı**
Paylaşım	—	Paylaşılabilir	Paylaşılmaz
Örnek	Dog IS-A Animal	Employee HAS-A Address	Car HAS-A Engine
UML	Boş üçgenli ok	Boş elmas ◇	Dolu elmas ◆

PDF 6 alıştırma cevapları

S1. `class P { int x = 1; } class Q extends P { int y = 2; } Q q = new Q(); System.out.println(q.x + q.y);`

- Cevap: 3
- Neden: Q, x'i P'den miras alır; 1 + 2 = 3.

S2. `P'de hi() → "P", Q'da hi() → "Q". P p = new Q(); p.hi();`

- Cevap: Q
- Neden: Overriding: runtime'da object'in gerçek tipi (Q) belirler.

S3. `P p = new P(); System.out.println(p instanceof Q); System.out.println(new Q() instanceof P);`

- Cevap: false / true
- Neden: P bir Q değildir (parent, child değil). Q bir P'dir (IS-A).

S4. `Box box = new Box(); H h1 = new H(box), h2 = new H(box); h1.b.v = 5; System.out.println(h2.b.v);`

- Cevap: 5
- Neden: Aggregation: h1 ve h2 aynı Box object'ini paylaşır; biri değiştirince diğeri görür.

Ek çıktı tahmini soruları

Tüm cevaplar Java 21 ile çalıştırılarak doğrulandı. Önce kendin çöz, sonra cevaba bak.

Temeller (PDF 1-3)

S1. `int x = 7; System.out.println(x++ + ++x);`

- Cevap: 16
- Neden: x++ → 7 kullanılır, x = 8. ++x → x = 9, 9 kullanılır. 7 + 9 = 16.

S2. `System.out.println(1 + 2 + "3" + 4 + 5);`

- Cevap: 3345
- Neden: Soldan sağa: 1 + 2 = 3 (sayı), sonra "3" ile String olur → "33", ardından 4 ve 5 metne eklenir.

S3. `int s = 0; for (int i = 1; i <= 10; i++) { if (i % 2 == 0) continue; if (i > 7) break; s += i; }`

- Cevap: 16
- Neden: Çiftler atlanır; 1 + 3 + 5 + 7 = 16. i = 9'da break.

S4. `int k = 3; switch (k) { case 1: print("a"); case 3: print("c"); case 4: print("d"); break; default: print("z"); }`

- Cevap: cd
- Neden: case 3'ten başlar, break olmadığı için case 4'e düşer, oradaki break'te durur.

S5. `int n = 0; while (n < 10) n += 4; System.out.println(n);`

- Cevap: 12
- Neden: 0 → 4 → 8 → 12; 12 < 10 false.

S6. `System.out.println((int) 3.99 + Math.round(3.5));`

- Cevap: 7
- Neden: (int) kesirli kısmı atar → 3; Math.round(3.5) yuvarlar → 4.

S7. `short sh = 10; sh += 5.7; System.out.println(sh);`

- Cevap: 15
- Neden: Compound assignment gizli cast içerir: sh = (short)(10 + 5.7) = (short) 15.7 = 15.

S8. `System.out.println(17 % 5 + -17 % 5);`

- Cevap: 0
- Neden: 17 % 5 = 2; −17 % 5 = −2 (işaret soldan gelir). 2 + (−2) = 0.

S9. `String s1 = "Hi", s2 = new String("Hi"); System.out.println((s1 == s2) + " " + s1.equals(s2));`

- Cevap: false true
- Neden: == adres karşılaştırır (new ayrı object), equals içerik.

S10. `System.out.println(10 >> 1 | 1);`

- Cevap: 5
- Neden: Shift önce (öncelik 5), bitwise OR sonra (8). 10 >> 1 = 5 (0101); 0101 | 0001 = 0101 = 5.

S11. `int i = 0; do { i += 3; } while (i < 0); System.out.println(i);`

- Cevap: 3
- Neden: do-while body en az bir kez çalışır; sonra 3 < 0 false.

OOP (PDF 4-6)

S12.

```
class Animal { Animal() { System.out.print("A"); } void sound() { System.out.print("..."); } }
class Dog extends Animal { Dog() { System.out.print("D"); } @Override void sound() { System.out.print("Hav"); }
}
Animal an = new Dog();
an.sound();
```

- Cevap: AD sonra Hav
- Neden: new Dog() önce super() ile Animal constructor'ı (A), sonra Dog (D). sound() overriding → object tipi Dog → Hav.

S13.

```
class Parent { static void s() { print("PS "); } void i() { print("PI "); } }
class Child extends Parent { static void s() { print("CS "); } @Override void i() { print("CI "); } }
Parent p = new Child();
p.s(); p.i();
```

- Cevap: PS CI
- Neden: static method override edilmez (hiding) → reference tipi Parent → PS. Instance method override → object tipi Child → CI.

S14. `class Cnt { int v; Cnt add(int n) { v += n; return this; } } System.out.println(new Cnt().add(2).add(3).v);`

- Cevap: 5
- Neden: return this ile method chaining; aynı object'e 2 ve 3 eklenir.

S15. `class St { static int c; int i; St() { c++; i++; } } new St(); St t2 = new St(); System.out.println(St.c + " " + t2.i);`

- Cevap: 2 1
- Neden: c static → iki object ortak artırır (2). i instance → her object kendi i'sini 0'dan artırır (1).

S16. `static int f(int x) { x = x * 2; return x; } int m = 5; int r = f(m); System.out.println(m + " " + r);`

- Cevap: 5 10
- Neden: Pass-by-value: m'nin kopyası değişir, m 5 kalır; dönen değer 10.

Hata mı, çalışır mı?

Kod	Sonuç	Neden
float f = 3.5;	**Compile error**	Ondalık literal double; 3.5f olmalı
long l = 15000000000;	**Compile error**	L eki yok → int sayılır, sığmaz
byte z = x + y; (byte x, y)	**Compile error**	x + y int olur; cast gerekir
boolean b = 1;	**Compile error**	boolean yalnızca true/false
if (x = 5) { }	**Compile error**	int boolean değil
switch (5L) { … }	**Compile error**	long switch'te desteklenmez
class C extends A, B { }	**Compile error**	Class'larda multiple inheritance yok
Point p = new Point(); (yalnız Point(int,int) var)	**Compile error**	Constructor yazılınca default eklenmez
this(w); ikinci satırda	**Compile error**	this() ilk statement olmalı
static method içinde instance field'a erişim	**Compile error**	non-static variable … static context
final class'ı extend etmek	**Compile error**	cannot inherit from final
protected method'u default ile override etmek	**Compile error**	weaker access privileges
int t; System.out.println(t); (local)	**Compile error**	might not have been initialized
10 / 0	**Runtime: ArithmeticException**	Tamsayı sıfıra bölünemez
10.0 / 0	Çalışır: **Infinity**	Ondalık bölmede istisna yok
Integer.MAX_VALUE + 1	Çalışır: **−2147483648**	Sessiz overflow
(Dog) new Cat() (Animal ref.)	**Runtime: ClassCastException**	Yanlış tipe downcast
null String'de .length()	**Runtime: NullPointerException**	null üzerinde method
arr[arr.length]	**Runtime: ArrayIndexOutOfBoundsException**	Son index length − 1

Sık yapılan hatalar

Altı PDF'te geçen tüm "sık hata" ve "dikkat" slaytlarının toplu listesi.

Variables, types, operators (PDF 1)

Hata	Doğrusu
Initialize edilmemiş local variable kullanmak	Local variable default değer almaz; önce değer ver
float f = 3.5;	3.5f (ondalık literal double'dır)
Büyük long'da L ekini unutmak	15_000_000_000L
5 / 2 → 2.5 sanmak	Integer division → 2. Ortalama için (double) sum / n
Integer.MAX_VALUE + 1 hata verir sanmak	Sessiz overflow → negatif sayı; gerekirse long
String'i == ile karşılaştırmak	Her zaman equals() (harf duyarsız: equalsIgnoreCase())
if (x = 5)	Karşılaştırma == ile; Java'da compile error
0.1 + 0.2 == 0.3	false; ondalıklar == ile karşılaştırılmaz, para için BigDecimal
s = s + 5 (short)	int olur; s += 5 veya (short) cast
(int) 9.99 → 10 sanmak	9 (truncate); yuvarlama Math.round()

if ve switch (PDF 2)

Hata	Doğrusu
if (...);	Noktalı virgül boş statement olur; alttaki block hep çalışır
{ } olmadan iki satır	Sadece ilk satır if'e bağlı; girinti anlam taşımaz
Ladder'da yanlış sıra (>= 50 önce, >= 90 sonra)	En kısıtlayıcı koşul önce; logic error, derleyici uyarmaz
switch'te break unutmak	Fall-through: sonraki case'ler de çalışır
switch'te long/float/double/boolean	Desteklenmez → compile error
case'te variable veya aralık (case > 10)	case değeri constant ve unique olmalı
null String ile switch	NullPointerException
String switch'te harf farkı	Büyük/küçük harf duyarlı; toLowerCase()

Loop'lar (PDF 2-3)

Hata	Doğrusu
i <= arr.length (off-by-one)	i < arr.length; aksi ArrayIndexOutOfBoundsException
for (...);	Body boş olur; alttaki block bir kez çalışır
Yanlış yönde update (i--)	Sonsuz loop
for değişkenini loop dışında kullanmak	Scope dışı; gerekiyorsa loop'tan önce tanımla
while'da update'i unutmak	Sonsuz loop; Ctrl+C ile durur
while'da continue'dan sonra update	i++ atlanır → sonsuz loop; update'i continue'dan önce yap
Nested while'da inner sayacı dışarıda başlatmak	Inner sadece ilk satırda çalışır; outer'ın içinde sıfırla
do-while'da sondaki ; unutmak	while (condition); zorunlu
for-each ile array değiştirmeye çalışmak	Loop variable kopya; klasik for + arr[i]
for-each sırasında Collection'dan silmek	ConcurrentModificationException
Switch içindeki break'in loop'u bitirdiğini sanmak	Sadece switch'ten çıkar; labeled break kullan
Sentinel'de count = 0 kontrolsüz ortalama	0/0 → NaN; if (count > 0)

Comments (PDF 3)

Hata	Doğrusu
İç içe /* /* */ */	Compile error; multi-line iç içe yazılamaz
Kodu tekrar eden comment (x'e 5 ata)	NEDEN'i açıkla; anlamlı isim kullan
Kod değişti, comment eski kaldı	Comment'i de güncelle

Class, method, constructor (PDF 4)

Hata	Doğrusu
Constructor'a void yazmak	Method olur; new ile çalışmaz
Parametreli constructor varken new Point()	Default kaybolur; no-arg constructor'ı da yaz
p2 = p1'i kopya sanmak	Aynı object (aliasing); copy constructor ile yeni object
Method içinde primitive parametreyi değiştirip orijinalin değiştiğini sanmak	Pass-by-value: kopya değişir
static method'dan instance method'u object olmadan çağırmak	Compile error; önce object oluştur
void olmayan method'da bazı yollarda return unutmak	Her yol değer döndürmeli
Convention'a uymayan isimler (student_record, Print_Info)	StudentRecord, printInfo(), MAX_SIZE

static, this, overloading, overriding, super (PDF 5)

Hata	Doğrusu
Constructor'da this olmadan rollno = rollno	Shadowing; field'lar 0/null kalır. this.rollno = rollno
this() / super() ilk satırda değil	Compile error; ilk statement olmalı
this() ve super() aynı constructor'da	Birlikte kullanılamaz
static method'da this/super	Kullanılamaz
s1.count = 99 ile static'e erişmek	Çalışır ama önerilmez; Student.count
Sadece return type farklı overload	Compile error: already defined
sum(int,long) / sum(long,int) + sum(20, 20)	Ambiguous; argümanı açıkça belirt (20L)
@Override yazmamak (Run ≠ run)	Sessizce yeni method; @Override hatayı yakalar
Override'da erişimi daraltmak	weaker access privileges hatası
static method'u override ettiğini sanmak	Method hiding; reference tipine göre çalışır
Parent'ta no-arg yokken örtük super()	Compile error; super(name) çağır
Field'ın polymorphic olduğunu sanmak	Field reference tipine göre çözülür

Inheritance ve aggregation (PDF 6)

Hata	Doğrusu
class C extends A, B	Multiple inheritance yok; interface kullan
Subclass'tan private field'a doğrudan erişim	Getter veya protected
Kardeş class method'unu çağırmak (Cat → bark())	Kardeşler birbirini görmez
instanceof kontrolsüz downcast	ClassCastException
Upcast reference ile subclass method'u çağırmak	Reference tipi izin vermez; downcast et
final class'ı extend / final method'u override	Compile error
Car extends Engine (IS-A olmayan inheritance)	HAS-A: private Engine engine = new Engine();
Çok derin hiyerarşi	Sığ tut; composition tercih et
İki interface aynı default method → override etmemek	Compile error; override edip A.super.hello() ile seç

Son tekrar listesi

PDF 1

- 3 variable: local (default **yok**), instance (object başına), static (tek kopya). Global variable yok.
- 8 primitive: byte, short, int, long, float, double, char (16 bit Unicode), boolean. Reference: String, array, class, interface, enum (default null).
- Ondalık literal **double**, float için f; büyük long için L. Overflow sessiz.
- Widening otomatik; narrowing **(type)** cast ister, kesirli kısmı atar. boolean cast edilemez. byte/short/char aritmetikte int'e yükselir.
- int / int → kesir atılır; 10 / 0 ArithmeticException, 10.0 / 0 Infinity.
- x++ eski değer, ++x yeni değer. && ve || short-circuit; & ve | değil.
- a op= b gizli cast içerir. Assignment ve ternary sağdan sola.
- String: == adres, **equals()** içerik.

PDF 2

- Koşul boolean olmalı; ladder'da en kısıtlayıcı koşul önce; her zaman { }.
- switch: long/float/double/boolean **yok**; case constant ve unique; break yoksa fall-through; Java 14+ arrow'da break gerekmez.
- for: init bir kez → condition → body → update. Tekrar sayısı biliniyorsa.
- for-each: index yok, eleman kopyası, array'i değiştiremez.

PDF 3

- while: önce kontrol (0+ tur, entry-controlled). do-while: sonra kontrol (**1+ tur**, exit-controlled, sonda ;).
- break loop/switch'i bitirir; continue turu atlar; return method'u bitirir. Labeled hâli dış loop'u hedefler.
- // tek satır, /* */ çok satır (iç içe olmaz), /** */ Javadoc. Comment NEDEN'i anlatır.

PDF 4

- Object = state + behavior + identity. Class = blueprint. 4 temel: inheritance, polymorphism, abstraction, encapsulation.
- Hedef: **düşük coupling, yüksek cohesion**.
- Aggregation (zayıf, dışarıdan) ↔ composition (güçlü, içeride).
- Naming: Class PascalCase isim, method camelCase fiil, CONSTANT, package küçük harf.
- new heap'te object; variable reference tutar; p2 = p1 aliasing.
- Signature = ad + parametreler. Java **pass-by-value**.
- Access: public > protected > default > private.
- Constructor: adı class adı, return type yok; herhangi biri yazılınca default eklenmez; inherit edilmez.

PDF 5

- static: class'a ait tek kopya; static method instance üyeye doğrudan erişemez; this/super kullanamaz; override edilmez (hiding).
- Sıra: static block → main → instance block → constructor.
- this: field ayırt etme, this() chaining (ilk satır), return this ile method chaining.
- Overloading: aynı isim, farklı parametre (sayı/tip/sıra), compile-time. Sadece return type farkı yetmez.
- Overriding: aynı imza, subclass'ta, runtime; erişim daraltılamaz; final/private/static/constructor override edilmez. @Override kullan.
- super: super.field, super.method(), super(...) ilk satır, yazılmazsa compiler ekler. Constructor'lar parent'tan child'a çalışır.

PDF 6

- extends ile **tek** class'tan türetme; private ve constructor doğrudan miras alınmaz; her class Object'ten türer.
- Single, multilevel, hierarchical class'larla; multiple (ve onu içeren hybrid) yalnızca interface'lerle (diamond problem).
- Upcasting otomatik; downcasting instanceof ile güvenli; yanlış cast ClassCastException.
- final class extend edilemez (String, Integer, Math); final method override edilemez.
- IS-A cümlesi doğru değilse inheritance yerine **HAS-A**.